

Jashore University of Science and Technology

Jashore-7408, Bangladesh



Campus Based Donation Management System

Submitted By

Ramjan Ali Kha

Student ID: 200114

Registration Year: 2020-21

Supervisor

Dr. Mohammad Nowsin Amin Sheikh

Assistant Professor

Department of Computer Science and Engineering

This project report was submitted in partial fulfillment of the

Requirements for the BSc.Engg Degree

In Computer Science and Engineering.

April 2026

Jashore University of Science and Technology

Jashore-7408, Bangladesh



This project titled **Campus Based Donation Management System** submitted by **Ramjan Ali Kha** for final defence for the degree of B.Sc. in Computer Science and Engineering on April 27, 2025.

Signature of the Student

Ramjan Ali Kha

Signature of the Supervisor

Dr. Mohammad Nowsin Amin Sheikh

Acknowledgments

I want to thank **Almighty Allah**, the Creator and the Sustained of the word, for providing us with the opportunity, capacity, energy, spirit, and patience to finish the report.

I want to express my sincere gratitude to Dr. Mohammad Nowsin Amin Sheikh, assistant professor at Jashore University of Science and Technology and my respected educator and supervisor. His unwavering advice and direction were crucial to the success of this project. In order to make this project writing as perfect as possible, he was constantly offering valuable assistance. The completion of the report would not have been possible without his extensive knowledge, insightful ideas, boundless enthusiasm, invaluable advice, assistance, and suggestions. I am honored to have had the chance to work and study under his supervision.

I also want to express my gratitude to the honorable faculty of the Department of Computer Science and Engineering. I appreciate their support and helpful criticism, which enabled me to write my project report more skillfully.

My friends and family have been a major source of inspiration and passion for me, and for that I am incredibly grateful. All of their love, kindness, and support have been invaluable to me during this time.

I would want to thank my parents and family for their unwavering support and motivation. I am grateful for your understanding, love, support, and patience, all of which inspired me to complete my studies.

Table of Contents

Acknowledgments	i
Abstract	x
Chapter 1: Introduction	2
1.1 Overview	2
1.2 Background	3
1.3 Motivation	5
1.4 Problem Statement	6
1.5 Questions Addressed by the Project	7
1.6 Objectives	9
1.7 Organization	10
Chapter 2: Background and Related Technologies	12
2.1 Overview	12
2.2 Conventional Donation Management Techniques	12
2.3 Existing Digital Donation Management Solutions	13
2.3.1 Critical Limitations in Existing Solutions	14
2.4 Role-Based Access Control in Management Systems	15
2.5 Robust Payment Gateway Architectures	16
2.6 Advanced Sentiment Analysis and Testimonial Moderation	17
2.7 Modern Web Technologies	18
2.8 Advanced Features and Capabilities	19
2.8.1 Physical Donation Management	19
2.8.2 Expense Voucher and Financial Control Systems	19
2.8.3 Withdrawal and Reserve Fund Management	20
2.8.4 Machine Learning Predictive Analytics	21

2.8.5	Automated Communication Infrastructure	21
Chapter 3: Methodology		24
3.1	Overview	24
3.2	System Design	24
3.2.1	Key Architectural Goals	24
3.2.2	Frontend Design	25
3.2.3	Backend Design	26
3.2.4	Database Design	27
3.3	Development Process	27
3.3.1	Frontend Development	27
3.3.2	Backend Development	28
3.4	Testing	29
3.5	Deployment Strategy	31
3.6	Security Measures	31
Chapter 4: System Architecture and Design		34
4.1	Overview	34
4.2	Frontend Architecture	34
4.3	Backend Architecture	34
4.4	Database Schema and Data Dictionary	35
4.4.1	Table Campaigns	35
4.4.2	Table Donations	36
4.4.3	Table Volunteer Reports	37
4.5	ER Diagram	39
4.6	Use Case Diagram	39
4.7	Schema Diagram	40
4.8	System Context Diagram	41

4.9	Container Diagram	42
4.10	Component Diagram	42
4.11	Class Diagram	43
4.12	Data Flow Diagram (Level 0)	44
4.13	Data Flow Diagram (Level 1)	45
4.14	Deployment Diagram	45
4.15	Activity Diagram: Donation Workflow	46
4.16	Campaign Lifecycle State Diagram	48
4.17	Authentication Flow	48
4.17.1	Registration, Login, and Token Issuance	49
4.17.2	Authorization and Access Control	49
4.18	Payment Processing and Ledger Integration	49
4.18.1	Sequence Diagram: Payment Callback Flow	50
4.18.2	How a Transaction Starts	50
4.18.3	Callback Handling and Idempotency	51
Chapter 5: System Implementation		53
5.1	System Setup	53
5.1.1	Technologies Used	53
5.2	System Implementation	54
5.2.1	User Authentication	54
5.2.2	Dashboard Overview	55
5.2.3	User and Role Management	56
5.2.4	Campaign Management	56
5.2.5	Donation Processing	57
5.2.6	Volunteer Management	58
5.2.7	Testimonial and Sentiment Analysis	58
5.2.8	Analytics and Reporting	59

5.2.9	Additional Feature Screenshots	60
5.2.10	Screenshot File Arrangement	63
5.3	Key Features Implementation	65
5.3.1	Donation Tracking and Processing	65
5.3.2	Campaign Updates and Communications	65
5.3.3	Volunteer Reporting and Recognition	65
5.3.4	Physical Donation Management	65
5.3.5	Expense Voucher Management	66
5.3.6	Financial Controls and Withdrawal Management	66
5.3.7	Payment Integration	66
5.3.8	Machine Learning Features	66
5.3.9	Email and SMS Notifications	68
5.3.10	Testimonial Moderation	68
Chapter 6:	System Evaluation and Results	70
6.1	System Evaluation Overview	70
6.2	Automated Test Coverage	70
6.3	Functional Validation Scenarios	70
6.4	ML Endpoint Validation	71
6.5	Limitations and What Comes Next	72
Chapter 7:	Conclusion and Future Work	74
7.1	Conclusion	74
7.2	Future Work	75
7.2.1	Suggested Roadmap	76
References		78

List of Figures

4.1	Entity-Relationship Diagram.	39
4.2	Use Case Diagram.	40
4.3	Database Schema Diagram.	41
4.4	System Context Diagram.	41
4.5	Container Diagram.	42
4.6	Component Diagram.	43
4.7	Class Diagram.	44
4.8	Data Flow Diagram (Level 0).	44
4.9	Data Flow Diagram (Level 1).	45
4.10	Deployment Diagram.	46
4.11	Activity Diagram for Donation Workflow.	47
4.12	Campaign Lifecycle State Diagram.	48
4.13	Sequence Diagram for Donation Payment Flow.	50
5.1	Login and Authentication Interface.	55
5.2	Admin Dashboard Overview.	55
5.3	User and Role Management Panel.	56
5.4	Campaign Creation and Management Interface.	57
5.5	Donation Checkout and Payment Initiation.	57
5.6	Volunteer Activity and Reporting Interface.	58
5.7	Testimonial Moderation and Sentiment Review.	59
5.8	Analytics and Reporting Dashboard.	59
5.9	Campaign Updates and Announcement Interface.	60
5.10	Physical Donation Verification and Tracking.	60
5.11	Expense Voucher Management and Approval Workflow.	61
5.12	Withdrawal Request and Approval Interface.	62
5.13	Reserve Fund Transparency.	62

5.14 Payment Transaction History and Status Tracking. 63

5.15 Machine Learning Insights and Prediction Outputs. 63

List of Tables

1	List of Acronyms.	ix
3.1	Frontend Component Categorization	26
3.2	Testing Matrix	30
3.3	Core Security Controls	32
4.1	Data Dictionary - Campaigns Entity	36
4.2	Data Dictionary - Donations Entity	37
4.3	Data Dictionary - Volunteer Reports	38
5.1	Frontend Technologies.	53
5.2	Backend Technologies.	54
5.3	Implementation Screenshot Naming Scheme	64
6.1	Implemented Evaluation Layers	70
6.2	Critical Workflow Validation	71
7.1	Future Enhancement Roadmap	76

List of Acronyms

Table 1: List of Acronyms.

Acronyms	Full Form
DMS	Donation Management System
UI	User Interface
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
JWT	JSON Web Token
ORM	Object-Relational Mapping
REST	Representational State Transfer
HTTPS	Hypertext Transfer Protocol Secure
SQL	Structured Query Language
EF	Entity Framework
RBAC	Role Based Access Control
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
TypeScript	Programming Language
React	JavaScript Library
Redux	State Management
.NET	Framework
C#	Programming Language

Abstract

Effective philanthropic administration demands the seamless coordination of fundraising campaigns, transparent allocation of resources, rigorous volunteer management, and accountable financial stewardship. Despite this, a significant number of charitable organizations, particularly those operating within academic institutions, continue to depend on paper-based record-keeping and fragmented manual workflows. The limitations of such approaches became especially evident during participation in two campus fundraising initiatives: the 2024 Sylhet flood relief campaign and the collection drive for the medical expenses of our late sister Atika (CSE 17-18). In both cases, the reliance on manual processes introduced data inaccuracies, obscured fund utilization, and imposed unnecessary procedural overhead. These direct observations provided the central motivation for the development of the Donation Management System (DMS) presented in this report. The DMS is a consolidated web-based platform designed to unify and automate the full spectrum of donation-related operations.

The system adopts a multi-tier architectural design. At the application layer, an ASP.NET Core Web API implements business logic and exposes RESTful service endpoints. The presentation layer is developed using React coupled with TypeScript, yielding a responsive and statically typed user interface. Persistent data storage is managed through Entity Framework Core atop a relational database, ensuring maintainable schema evolution as the project matures.

Security considerations have been embedded across every layer of the architecture. Password credentials are protected through BCrypt hashing, cross-site request forgery is mitigated via token-based safeguards, all input is subject to server-side validation, and monetary transactions are processed exclusively through a secured payment gateway. A responsive design strategy further ensures cross-device interoperability, from desktop workstations to mobile handsets. Collectively, these measures substantially reduce manual intervention, strengthen the transparency of donation tracking, and simplify the end-to-end management of fundraising operations.

The platform's feature set encompasses campaign lifecycle management, both online and physical donation processing, volunteer coordination, role-based access control, automated sentiment

analysis of donor testimonials, and in-kind contribution tracking with OTP-based verification. The principal objective is to cultivate donor confidence through operational visibility and to accelerate organizational workflows through targeted automation.

Chapter 1

Introduction

Chapter 1: Introduction

1.1 Overview

The **Campus Based Donation Management System** constitutes a web-based software architecture conceived to digitize and systematize the administrative processes governing institutional philanthropy [1, 2]. By unifying administrators, donors, and volunteers within a single digital ecosystem, the platform introduces role-differentiated interfaces that substantially mitigate the logistical inefficiencies characteristic of paper-dependent coordination models.

Functionally, the system addresses three principal operational domains:

- **Financial Administration:** Real-time monitoring of donation transactions, secure online payment facilitation through SSLCommerz [3], and physical donation authentication via One-Time Password (OTP) protocols to ensure verified recording of in-kind assets.
- **Operational Orchestration:** Structured lifecycle management of fundraising campaigns, meritocratic progression frameworks for volunteer recognition, and formalized expenditure voucher authorization chains that replace ad-hoc approval mechanisms.
- **Predictive Intelligence:** An integrated machine learning subsystem capable of forecasting donation volumes, detecting transactional anomalies, and estimating donor attrition probabilities. This enables proactive rather than reactive administrative decision-making.

From an architectural standpoint, the platform adheres to a contemporary three-tier design. The presentation tier employs **React with TypeScript** [4, 5], ensuring a responsive and type-safe user interface. Business logic resides within an **ASP.NET Core Web API** [6, 7], while data persistence is maintained through **Microsoft SQL Server** accessed via Entity Framework Core [8, 9]. System-wide security enforcement is achieved through JSON Web Token (JWT) authentication, BCrypt cryptographic hashing, and Role-Based Access Control (RBAC) [10]. Together, these components establish a resilient and extensible foundation amenable to iterative feature expansion.

1.2 Background

Charitable organizations and campus-based donation initiatives have historically operated through manual, paper-oriented systems, or at best, loosely connected digital tools, to track contributions, coordinate campaigns, and manage volunteer personnel [2, 11]. While such approaches may suffice at small scales, they become increasingly untenable as operational complexity grows. The following deficiencies, identified through both literature review and practical observation, directly informed the requirements specification for the proposed DMS:

- **Tracking Latency:** Substantial delays impede the real-time monitoring of donation inflows and campaign advancement. Status information often lags behind actual conditions by hours or days, hindering timely decision-making.
- **Data Redundancy:** Parallel entry of identical records across disconnected spreadsheets introduces systematic inconsistencies and clerical errors. Post-hoc data cleansing is both time-intensive and prone to compounding inaccuracies.
- **Operational Inefficiency:** Volunteer contributions remain largely unquantified in the absence of structured tracking mechanisms. No standardized method exists for recording hours worked, tasks completed, or resource utilization.
- **Analytical Deficiencies:** Without dedicated analytical tools, evaluating campaign performance, identifying donor behavioral trends, or detecting declining engagement becomes prohibitively difficult.
- **Obstacles to Communication:** Bidirectional communication channels between organizations and their donor constituencies are frequently either informal or nonexistent, resulting in inadequate feedback loops.
- **Lack of Transparency:** When fund utilization remains opaque to contributors, institutional trust diminishes progressively, reducing the likelihood of sustained donor partici-

pation in subsequent initiatives.

- **No Financial Accountability:** Audit trails are commonly incomplete or altogether absent, precluding reliable verification that disbursed funds align with stated organizational objectives.
- **Limited Donor Visibility:** Donors are seldom afforded access to real-time performance indicators regarding campaign trajectories or expenditure breakdowns, which measurably discourages repeat contributions.
- **Absence of Physical Donation Management:** In-kind contributions (such as clothing, food, and supplies) are frequently undocumented. The resulting gaps in asset records compromise institutional accountability.
- **Inadequate Expense Voucher Oversight:** Campaign-related expenditures are not governed by formal voucher authorization workflows, complicating the verification and validation of operational costs.
- **Unregulated Fund Withdrawal Processes:** The absence of structured disbursement protocols and controlled reserve fund management elevates the risk of financial misappropriation and weakens audit trail integrity.
- **Absence of Predictive and Analytical Capabilities:** Conventional systems are inherently reactive; they archive historical data but provide no capacity for forecasting donation trends, identifying transactional outliers, or modeling donor attrition patterns.
- **Lack of Volunteer Recognition and Incentivization:** Without formalized merit-based progression and achievement recognition mechanisms, volunteer motivation and longitudinal retention are adversely affected.
- **Insufficient Automated Notification Infrastructure:** Critical communications, including contribution confirmations, verification credentials, and status updates, are transmitted manually, if at all, leading to delayed and unreliable stakeholder engagement.

The accelerating pace of digital transformation, coupled with rising stakeholder expectations for sophisticated online interfaces, has rendered the adoption of centralized and automated management architectures a pressing institutional necessity. The proposed Donation Management System addresses these documented shortcomings by providing a comprehensive digital framework that consolidates critical operations while upholding stringent standards of data security, financial transparency, and inclusive accessibility.

1.3 Motivation

The motivation underpinning this project stems from the recurring operational inefficiencies observed across traditional philanthropic administration workflows. Charitable organizations, particularly those embedded within academic settings, encounter persistent structural challenges across several interrelated domains:

- The concurrent management of heterogeneous donation channels (spanning cash, online payments, and mobile banking) without a unified system for reconciliation.
- The systematic tracking of volunteer engagement and the quantitative assessment of individual contributions to organizational impact.
- The rigorous evaluation of campaign performance, supplemented by structured mechanisms for capturing and analyzing donor sentiment.
- The timely dissemination of operational updates to stakeholders and beneficiaries regarding campaign progress and fund allocation.
- The maintenance of secure, auditable repositories for financial transaction records and sensitive donor information.

This project endeavours to construct a scalable and extensible software framework that addresses the aforementioned bottlenecks [12], while simultaneously establishing a stable foundation for future enhancements in philanthropic management. The adoption of contemporary web

technologies and adherence to established software engineering standards ensures operational reliability, robust security enforcement, and a high degree of user satisfaction [13].

1.4 Problem Statement

Organizations responsible for administering philanthropic initiatives face compound operational challenges arising from their dependence on manual processes and fragmented data management architectures [14, 2]. These challenges manifest across several critical dimensions:

- **Absence of Real-Time Monitoring:** Considerable information latency prevents both donors and administrators from tracking campaign trajectories and contribution statuses with any immediacy.
- **Manual Collection Vulnerabilities:** Non-automated donation procurement methods, whether cash-based or through informal transfer channels, are inherently inefficient, susceptible to clerical error, and present non-trivial security exposure.
- **Fragmented Volunteer Coordination:** The lack of a centralized management framework hampers the assignment of tasks, the tracking of individual engagement, and the empirical measurement of volunteer impact.
- **Data Integrity Degradation:** Manual record-keeping practices produce systematic data inconsistencies, undermining the reliability of reports generated for institutional oversight and decision-making.
- **Underdeveloped Analytical Capacity:** Administrative entities lack the diagnostic tooling required to derive actionable insights from donor behavioral data, campaign performance metrics, or stakeholder testimonial sentiment.
- **Security and Privacy Risks:** Conventional systems that rely on unencrypted storage media expose sensitive financial records and donor profiles to significant confidentiality and integrity threats [15, 16].

- **Institutional Opacity:** Prevailing frameworks seldom incorporate transparency mechanisms; empirical evidence links donor motivation to the visible verification of fund expenditure and tangible outcome reporting.
- **Absence of Integrated Fiscal Accounting:** Most localized systems lack bookkeeping modules capable of synchronizing donation inflows with formal financial ledger management.

The convergence of these deficiencies underscores the imperative for an automated, web-centric management architecture: one that provides a resilient, transparent, and accountable environment for the administration of philanthropic capital and mission-driven campaign operations.

1.5 Questions Addressed by the Project

The development of the proposed system is guided by the following research questions, each of which addresses an identified gap in current donation management practice:

- To what extent can a unified web-based platform enhance the efficiency of donation collection, tracking, and campaign management compared to fragmented manual approaches?
- What measurable advantages does role-based access control confer in multi-stakeholder donation management environments?
- How does real-time donation tracking influence campaign transparency and, by extension, donor satisfaction and trust?
- In what ways does the integration of a secure online payment gateway streamline the transactional aspects of the donation process?

- Can sentiment analysis applied to donor testimonials yield actionable insights into campaign impact and stakeholder sentiment?
- Which database management and security techniques prove most effective in safeguarding sensitive donor information within this operational context?

1.6 Objectives

Informed by the preceding research questions, the project pursues the following principal objectives:

- **To develop a unified web-based platform** that consolidates donation collection, campaign management, volunteer coordination, and reporting into a single integrated system.
- **To implement role-based access control** with differentiated functional permissions for Administrators, Donors, Volunteers, and Campaign Creators.
- **To provide real-time donation and campaign tracking** that delivers transparency and immediate feedback to all stakeholders.
- **To integrate secure payment processing** via the SSLCommerz gateway for reliable and efficient transaction handling.
- **To deliver an optimal user experience** through a responsive, intuitive interface accessible across heterogeneous device classes.
- **To ensure data security and integrity** through robust authentication, encryption, and server-side validation mechanisms.
- **To incorporate sentiment analysis** on donor testimonials as a quantitative indicator of campaign effectiveness and donor satisfaction.
- **To furnish comprehensive reporting and analytics** capabilities that support evidence-based decision-making and organizational performance monitoring.

1.7 Organization

This project report is structured into six chapters, each corresponding to a distinct stage of the development process. The organization is as follows:

Chapter 1 Introduction. Presents the project overview, contextual background, motivating factors, problem formulation, research questions, objectives, and report structure.

Chapter 2 Background and Related Technologies. Surveys existing donation management solutions, examines relevant technological foundations, and identifies the functional gaps that the proposed system aims to address.

Chapter 3 Methodology. Details the system design approach, development process, testing strategy, deployment considerations, and security implementation.

Chapter 4 System Architecture and Design. Describes the frontend and backend architectures, database schema, and accompanying design artifacts including ER, Use Case, Schema, and C4 diagrams.

Chapter 5 Implementation. Documents the system setup, technology configuration, and implementation of key functional modules, supplemented with interface screenshots.

Chapter 6 Conclusion and Future Work. Summarizes the project outcomes and proposes directions for future enhancement and extension.

Chapter 2

Background and Related Technologies

Chapter 2: Background and Related Technologies

2.1 Overview

The progressive digitization of organizational operations has had far-reaching implications for the philanthropic sector. Donation management practices have undergone a gradual but substantive transition, moving from paper-based record-keeping and manual coordination to software-driven platforms designed to centralize campaign oversight, volunteer administration, and financial tracking. This chapter undertakes a critical examination of the prevailing approaches to donation management, surveys the principal digital platforms currently available, and delineates the technological foundations upon which the proposed Donation Management System (DMS) is constructed. The analysis culminates in the identification of specific functional and architectural gaps that the DMS is designed to remediate.

2.2 Conventional Donation Management Techniques

Conventional approaches to donation administration remain predominantly dependent on manual processes: paper-based documentation, in-person collection, spreadsheet-based tracking, and informal communication channels [1, 17]. Though operationally familiar and initially cost-effective, these methods introduce a well-documented set of deficiencies as organizational scale increases:

- **Real-Time Monitoring Gaps:** The absence of automated data capture mechanisms results in persistent latency between actual donation activity and its reflection in organizational records. Campaign progress indicators may lag by hours or days.
- **Data Integrity Vulnerabilities:** Manual transcription is inherently susceptible to typographical errors, receipt misplacement, and duplicated entries, each of which degrades the reliability of downstream reporting.
- **Volunteer Coordination Inefficiencies:** Without centralized task assignment and

progress tracking tools, the allocation and supervision of volunteer personnel becomes logistically unwieldy.

- **Analytical Capacity Limitations:** Spreadsheet-based environments offer minimal support for pattern recognition in donor behavior or systematic evaluation of campaign effectiveness.
- **Security Exposure:** Unencrypted files stored on shared network drives or personal devices present tangible risks to the confidentiality and integrity of financial and personal data.

2.3 Existing Digital Donation Management Solutions

The current landscape of donation management software comprises several platform categories, each addressing different subsets of the operational requirements identified above. A critical survey of representative solutions reveals the following:

As-Sunnah Foundation: Operating within a faith-based humanitarian framework, this organization maintains a proprietary digital platform oriented toward the South Asian donor base. The infrastructure integrates mobile financial services (MFS), including bKash and Nagad, to facilitate expedient resource mobilization, while employing visual evidence-based transparency mechanisms to sustain donor confidence [18]. Despite its effectiveness within its specific operational context, the proprietary and non-extensible nature of the platform limits its applicability to other institutional settings.

Donorbox: A cloud-hosted fundraising platform that provides integrated payment processing, comprehensive donor profile management, and configurable donation form interfaces [14]. The system supports real-time transaction monitoring and automated donor communication workflows. Its capabilities are well-suited to organizations whose primary collection channel is online monetary donations, though it offers limited support for in-kind contribution tracking.

GiveWP: Architected as a modular WordPress plugin, GiveWP facilitates extensive form cus-

tomization and ensures seamless compatibility within WordPress-based content management ecosystems. Organizations operating outside the WordPress framework derive considerably less utility from this solution.

Network for Good: A comprehensive fundraising coordination platform encompassing donor administration, volunteer lifecycle management, and integrated analytical reporting [19]. While positioned as an end-to-end solution, the associated licensing costs may render it inaccessible to resource-constrained nonprofit entities.

Blockchain-Enabled Systems: Emerging distributed ledger technologies demonstrate the potential to establish transparent and immutable audit trails for philanthropic transactions [20]. However, the majority of blockchain-based donation systems remain at experimental stages of development and have not achieved sufficient maturity for routine operational deployment.

2.3.1 Critical Limitations in Existing Solutions

A comparative analysis of the aforementioned platforms reveals several persistent shortcomings that limit their suitability for campus-based philanthropic operations:

- **Institutional Customization Deficiencies:** Commercial donation platforms are predominantly designed for general nonprofit use cases and do not accommodate the specialized workflows characteristic of academic and campus environments, where students, faculty, and external donors interact through distinct operational channels.
- **Financial Accessibility Barriers:** Subscription-based pricing models and elevated licensing fees restrict access for student-managed organizations with constrained budgets [21].
- **Integration Constraints:** Established platforms frequently lack interoperability pathways with existing institutional systems, including student information databases and university financial software.

- **Analytical Superficiality:** Few commercial solutions offer predictive analytical capabilities such as donation trend forecasting or donor attrition modeling; most are limited to retrospective reporting.
- **In-Kind Donation Neglect:** Physical and in-kind contributions, which are a common feature of campus-based collection drives, are inadequately supported or entirely unaddressed by prevailing platforms.
- **Financial Governance Gaps:** Structured expense voucher authorization workflows and controlled fund withdrawal mechanisms are largely absent from existing solutions.

2.4 Role-Based Access Control in Management Systems

Role-Based Access Control (RBAC) constitutes a foundational authorization model for multi-stakeholder digital platforms, enabling granular permission management across distinct user populations [10]. Within the context of the DMS, RBAC serves several critical functions:

- **Information Security Enforcement:** Access to sensitive financial records and personal data is restricted on the basis of authenticated user classification, ensuring that exposure is limited to authorized roles [16].
- **User Experience Optimization:** Interface customization on a per-role basis presents only contextually relevant functionality, thereby reducing cognitive load and improving task efficiency.
- **System Integrity Preservation:** Middleware-enforced permission checks on every incoming request prevent unauthorized data modification, including cases where requests are crafted to circumvent client-side restrictions.
- **Audit Trail Facilitation:** Because each action is attributed to a specific user and role, a comprehensive and traceable record of system interactions is maintained by design.

The Donation Management System implements a four-tier RBAC hierarchy comprising the following user classifications:

- **System Administrators:** Granted comprehensive system access, encompassing user administration, platform configuration, financial oversight, and content moderation.
- **Donors:** Authorized to perform donation transactions, browse active campaigns, maintain personal profiles, submit testimonials, and retrieve receipts.
- **Volunteers:** Restricted to activity reporting, hour logging, individual progress tracking, and achievement visualization.
- **Campaign Creators:** Permitted to manage campaign lifecycles, publish progress updates, monitor fundraising trajectories, and facilitate stakeholder communication.

2.5 Robust Payment Gateway Architectures

The integration of a reliable online payment processing capability is an essential requirement for any modern philanthropic platform, necessitating robust encryption, rigorous transaction validation, and proactive fraud mitigation. SSLCommerz, a leading payment aggregator within the South Asian digital economy [3], was selected for the DMS on the basis of its comprehensive feature set:

- **Multi-Modal Payment Support:** The gateway accommodates a diverse range of payment instruments, including credit and debit cards, mobile financial services (bKash, Nagad), and internet banking, all consolidated through a unified integration interface.
- **Cryptographic Security Compliance:** End-to-end encryption and adherence to the Payment Card Industry Data Security Standard (PCI-DSS) ensure the confidentiality and integrity of all transaction data [15].

- **Asynchronous Transaction Verification:** Upon payment completion or failure, the gateway dispatches server-side callbacks that enable instantaneous synchronization of payment statuses without manual intervention.
- **Auditable Transaction Records:** Each transaction is logged at sufficient granularity to support financial reconciliation, regulatory compliance, and post-hoc audit procedures.

2.6 Advanced Sentiment Analysis and Testimonial Moderation

Sentiment analysis, a subfield of natural language processing (NLP), enables the automated extraction of evaluative indicators from unstructured textual data [22]. Applied to donor testimonials, this capability permits organizations to derive quantitative assessments of stakeholder satisfaction and campaign impact without manual content review. Within the DMS, sentiment analysis serves the following operational functions:

- **Campaign Impact Assessment:** Aggregated sentiment scores across testimonial corpora provide a quantitative measure of donor perception regarding specific campaigns and organizational initiatives.
- **Automated Content Moderation:** Testimonials classified with negative sentiment are automatically flagged for administrative review prior to public display, reducing moderator subjectivity and enhancing content quality assurance.
- **Longitudinal Satisfaction Monitoring:** Tracking sentiment distributions over time facilitates the early detection of declining donor satisfaction or emerging reputational risks.
- **Decision Support Integration:** Sentiment metrics, when correlated with financial and operational data, provide administrators with a more comprehensive evaluative framework for campaign strategy refinement.

The DMS enforces a mandatory administrative review gate for negatively classified testimonials before publication, thereby maintaining public-facing content integrity without imposing the overhead of exhaustive manual screening [13].

2.7 Modern Web Technologies

The technological selections for the DMS were governed by considerations of framework maturity, community support breadth, and suitability for the target problem domain [12]:

- **Frontend:** React with TypeScript [4, 5, 23] provides the presentation layer, with Redux Toolkit managing centralized application state and Tailwind CSS handling responsive styling.
- **Backend:** ASP.NET Core Web API serves as the server-side framework, structured around RESTful service conventions with clearly separated controller and service layers.
- **Database:** Microsoft SQL Server provides relational data persistence, with Entity Framework Core serving as the Object-Relational Mapping (ORM) layer to abstract database operations into typed C# code [8, 9].
- **Authentication:** Stateless session management is implemented through JWT bearer tokens, complemented by OTP-based verification delivered via email and SMS for secondary authentication [24, 25].
- **Security:** Password credentials are secured via BCrypt hashing [26], form submissions incorporate CSRF token protection [27], and all user-supplied inputs undergo server-side validation and sanitization prior to database interaction [15].

The selection of these technologies ensures long-term system scalability and maintainability, supported by active open-source ecosystems and well-documented upgrade pathways [28].

2.8 Advanced Features and Capabilities

Beyond foundational donation collection and campaign management, the DMS incorporates several advanced functional modules that address persistent gaps in existing commercial platforms. These capabilities, described below, represent the principal areas of differentiation for the proposed system.

2.8.1 Physical Donation Management

The tracking and verification of in-kind and physical donations constitutes a notable omission in most conventional donation management platforms. The DMS addresses this gap through a dedicated tracking module [17]:

- **OTP-Based Donor Authentication:** Prior to the recording of any physical donation, the donor undergoes identity verification through a one-time password delivered via SMS, mitigating the risk of fraudulent or erroneous entries.
- **Structured Asset Documentation:** Each donated item is systematically catalogued, including descriptive metadata, estimated valuations, and temporal tracking information.
- **Audit Trail Maintenance:** Every physical donation event generates a verifiable record suitable for retrospective financial and compliance auditing.
- **Financial Ledger Integration:** In-kind contributions are synchronized with the monetary donation ledger, ensuring that aggregate campaign impact metrics remain comprehensive and accurate.

2.8.2 Expense Voucher and Financial Control Systems

Structured expenditure accounting and formalized voucher authorization workflows constitute essential financial governance mechanisms within the DMS:

- **Voucher Submission and Authorization:** All campaign-related expenditures require formal documentation and hierarchical approval prior to fund disbursement, preventing unilateral spending decisions.
- **Attachment Management:** Supporting documentation (receipts, invoices, and procurement records) can be uploaded and associated with individual voucher entries for verification purposes.
- **Status Tracking:** Voucher progression through the approval workflow (submitted, under review, approved, rejected) is monitored in real time.
- **Automatic Ledger Synchronization:** Approved expenditures are automatically reflected in the corresponding campaign financial records.

2.8.3 Withdrawal and Reserve Fund Management

Controlled fiscal disbursement mechanisms and transparent reserve fund administration reinforce institutional financial accountability:

- **Formal Withdrawal Request Workflows:** Fund disbursement follows a structured request-and-approval pipeline, ensuring that no withdrawal proceeds without administrative authorization.
- **Public Reserve Fund Dashboard:** A transparency-oriented dashboard displays aggregate reserve fund balances and recent transactional activity, accessible to all stakeholders.
- **Institutional Safeguards:** Configurable constraints and comprehensive audit trails guard against unauthorized fund depletion.
- **Automated Notification Infrastructure:** Relevant parties receive automated email notifications upon withdrawal approval or reserve fund status changes.

2.8.4 Machine Learning Predictive Analytics

The integration of machine learning capabilities enables evidence-based strategic planning and proactive risk identification [29, 30]:

- **Donation Demand Forecasting:** Singular Spectrum Analysis (SSA) time-series methodologies project future donation volumes across configurable temporal horizons, informing campaign resource planning [31].
- **Donor Attrition Prediction:** Binary classification algorithms identify donors exhibiting elevated disengagement risk based on recency, frequency, and monetary contribution features.
- **Anomaly Detection:** Statistical outlier detection methods identify transactional irregularities and atypical activity patterns within campaign donation streams [32, 33].
- **Campaign Strategy Optimization:** Insights derived from predictive models feed directly into campaign planning, enabling more informed allocation of organizational effort and resources.

2.8.5 Automated Communication Infrastructure

Timely and reliable stakeholder communication is essential for sustaining engagement and institutional credibility. The DMS automates the principal categories of outbound messaging:

- **Email Notifications:** Automated dispatch of donation receipt confirmations, campaign progress updates, and administrative decision notifications.
- **SMS Communications:** OTP delivery for donor verification and transmission of critical system alerts through SMS channels.

- **Event-Driven Triggers:** The notification subsystem operates on an event-driven architecture, ensuring that communications are dispatched in temporal alignment with the triggering activities.

Chapter 3

Methodology

Chapter 3: Methodology

3.1 Overview

This chapter walks through the practical steps taken to design, build, test, and secure the Donation Management System. Rather than following a rigid waterfall process, the work moved through repeated cycles by gathering requirements first then building features in small increments, testing them, and circling back to fix whatever came up. That iterative rhythm shaped most of the decisions described in the sections that follow.

3.2 System Design

At its core, the system is split into three distinct layers: a frontend client that users interact with, a backend API that handles business logic and data processing, and a relational database sitting underneath for persistent storage. Keeping these layers separate turned out to be especially useful when adding new features, since changes to one layer rarely forced changes in the others [12].

3.2.1 Key Architectural Goals

A few non-negotiable requirements were established before any code was written:

- **Modularity:** New capabilities such as a blockchain-based tracking module should plug in without disrupting what already works.
- **Fault Tolerance:** Payment gateways time out, databases occasionally deadlock, and the backend needed to handle both situations without crashing or leaving data in a half-updated state.
- **Scalability:** Campaign loads can spike dramatically during disaster relief periods. The web server and database tiers were designed so that each one could scale up independently of the other.

3.2.2 Frontend Design

We chose to build the user-facing side as a Single Page Application (SPA) using React paired with TypeScript [5]. TypeScript catches a whole class of bugs at compile time, which saved a lot of debugging effort later on. The main design choices were:

- **Responsive Layouts:** The UI relies on CSS Flexbox, Grid, and Tailwind CSS utility classes with media queries, so the same interface adjusts cleanly across phones, tablets, and desktops.
- **Reusable Components:** Instead of writing one-off UI code, we created shared React components for data tables, modals, and chart wrappers. This cut down on duplicated code considerably and made the interface feel more consistent.
- **Centralized State with Redux:** Redux Toolkit handles global state, with the store divided into feature slices like *authSlice*, *donorSlice*, and *adminSlice* [23]. Thunk middleware manages asynchronous operations such as API calls, which keeps the components themselves simpler.
- **Accessibility (a11y):** WCAG guidelines were followed throughout. That meant adding ARIA attributes where needed, offering a high-contrast theme option, and making sure every interactive element is reachable through keyboard navigation alone.

Table 3.1 gives a quick overview of how the frontend codebase is organized into layers.

Table 3.1: Frontend Component Categorization

Component Layer	What It Does
Presentation Components	Stateless pieces responsible only for rendering buttons, text, cards, and similar visual elements.
Container Components	Stateful wrappers that talk to APIs, subscribe to global state, and coordinate side effects.
Routing Module	Built on React Router DOM; handles page navigation and protects routes that require authentication.
Utility Services	Shared helpers for formatting (dates, currency) and an Axios-based HTTP interceptor layer.

3.2.3 Backend Design

On the server side, we went with ASP.NET Core Web API, following REST conventions for endpoint design [7]. The architecture breaks down into several clearly separated layers:

- **Controllers:** These sit at the boundary of the system where they validate incoming requests, route them to the right service, and format the response. Each major feature area (authentication, campaigns, donations, payments, testimonials, volunteers, vouchers, withdrawals) has its own controller.
- **Services:** All the actual business logic lives here. JWT generation, payment gateway communication, email and SMS dispatch, volunteer ranking calculations, and ML predictions are all handled by dedicated service classes.
- **Data Transfer Objects (DTOs):** API request and response shapes are defined separately from the database entities. This keeps the API surface stable even when the underlying schema changes.

- **Middleware:** Cross-cutting concerns such as JWT authentication, role-based authorization, CORS policy, and serving uploaded files are configured in the ASP.NET Core middleware pipeline.

3.2.4 Database Design

Microsoft SQL Server was selected as the database engine, and Entity Framework Core serves as the ORM [8, 9]. Several design decisions keep the data layer reliable:

- **Core Tables:** Users, Campaigns, Donations, Testimonials, VolunteerProfiles, VolunteerReports, Vouchers, ReserveFunds, and Withdrawals form the main schema.
- **Referential Integrity:** Foreign keys are defined explicitly, and delete behaviors (cascade, restrict, or set-null) are chosen per relationship to prevent orphaned records.
- **Monetary Precision:** All financial columns use `decimal(18,2)` to avoid floating-point rounding errors in accounting.
- **Indexed Lookups:** Frequently queried fields like user email are indexed so that lookups remain fast even as the dataset grows.

3.3 Development Process

Development was organized around Agile sprints [34, 35]. Each sprint lasted roughly two weeks, with a review at the end to decide what to prioritize next. This made it easy to adjust course when early testing revealed gaps in the original plan.

3.3.1 Frontend Development

Separate interfaces were built for each user role:

- **Admin:** A dashboard with user management controls, analytics views, and campaign oversight tools.

- **Donor:** Pages for making donations, writing testimonials, and managing profile details.
- **Volunteer:** Screens for logging activities, tracking hours, and viewing progress within assigned campaigns.

3.3.2 Backend Development

The backend grew module by module. Here is a summary of what each module covers:

- **Authentication and Authorization:** Stateless JWT-based login, email verification links, OTP-based second-factor authentication, and role-aware access control [25].
- **Donation Processing:** End-to-end donation workflows from creating the donation record through SSLCommerz payment to generating a receipt and updating campaign totals in real time.
- **Campaign Management:** Standard CRUD operations plus progress tracking, milestone markers, published updates, and tools for coordinating volunteers assigned to a campaign.
- **Physical Donation Handling:** Recording in-kind donations with item descriptions and estimated values, verifying the donor via SMS-based OTP, and syncing everything into the financial ledger.
- **Voucher and Expense Controls:** A voucher submission and approval pipeline, complete with document attachments and status tracking at each approval stage.
- **Withdrawal and Reserve Funds:** Formal withdrawal requests with admin approval, a publicly visible reserve fund dashboard, and safeguards against unauthorized fund depletion.
- **Sentiment Analysis:** Automated classification of testimonial text into positive, neutral, or negative categories, with mandatory admin review for anything flagged as negative.

- **Machine Learning:** Donation demand forecasting using Singular Spectrum Analysis, donor churn prediction, anomaly detection on donation streams, and campaign success estimation.
- **Analytics:** Aggregated views of donation data, campaign performance, volunteer contributions, and donor demographics with all of it exposed through dedicated API endpoints.
- **Notifications:** Email confirmations for donations, SMS-based OTPs, campaign update alerts, and event-triggered administrative notifications.

3.4 Testing

Both the frontend and backend were tested through a combination of automated test suites and hands-on scenario checks. The primary goal was to make sure that business logic worked correctly, API responses matched expectations, and the most important user flows did not break between releases.

Table 3.2: Testing Matrix

Test Level	Frameworks	Objective
Frontend Unit/Component	Vitest, React Testing Library	Check individual components, Redux slices, and client-side service functions in isolation.
Frontend Contract	MSW	Confirm that API wrapper functions send the right requests and handle responses correctly, using mocked endpoints.
End-to-End (E2E)	Playwright	Run through complete browser workflows like logging in, browsing campaigns, and stepping through the donation checkout.
Backend Unit/Integration	xUnit, Moq	Exercise controller and service methods under both normal and error conditions.
Security Regression	Manual API tests	Probe authorization boundaries, try malformed inputs, and verify payment callback validation.

Test code lives in well-defined locations within the project:

- **Frontend tests:** Found under `frontend/src/components/__tests__`, `frontend/src/store/slices/__tests__`, and `frontend/src/services/__tests__`.
- **Backend tests:** Located in `backend/DonationManagementSystem.Tests`, covering both unit-level and integration-level scenarios.
- **Script runners:** NPM scripts group test execution into categories such as unit, contract, coverage, and E2E so they can be run selectively.

Before any release build, a lightweight PowerShell script runs backend and frontend checks locally to catch regressions early.

3.5 Deployment Strategy

Frontend and backend are packaged and deployed separately, which gives flexibility to host them on the same server or split across different environments:

- **Frontend:** Vite produces an optimized static bundle that can be served by any standard web server.
- **Backend:** The ASP.NET Core API is published as a self-contained service; runtime settings come from appsettings files and environment variables.
- **Database:** Schema changes are handled via EF Core migrations, with automatic checks on application startup to ensure the database is up to date.
- **Environment Configuration:** Payment gateway credentials, JWT signing keys, and notification service settings are all injected through environment-specific configuration, keeping secrets out of source control.

3.6 Security Measures

Security was not treated as an afterthought; controls were built into every layer from the start, guided by OWASP recommendations [15] and general information security best practices [16].

Table 3.3: Core Security Controls

Threat	How It Is Mitigated
Credential compromise	Passwords are hashed with BCrypt before storage; sessions rely on short-lived JWT tokens [26, 25].
Privilege escalation	Every protected endpoint checks the caller's role claims before executing [10].
Payment callback tampering	Callback payloads from SSLCommerz are validated server-side before any financial records are updated.

A more detailed breakdown of the security layers:

- **Authentication:** JWT bearer tokens with configurable issuer, audience, and expiry windows [25]. Refresh tokens allow seamless session renewal without forcing a re-login.
- **Password Storage:** All passwords go through BCrypt hashing; password-reset tokens are verified against stored hashes as well [26].
- **Authorization:** RBAC policies restrict what each role can access so admin-only endpoints are unreachable for donor or volunteer accounts [10].
- **Input Validation:** Incoming data is validated server-side through DTO model annotations and controlled update flows, preventing injection and malformed-data attacks.
- **Network and Asset Security:** CORS is configured to accept requests only from known origins, and uploaded files are served from controlled static-file paths rather than arbitrary locations.

Chapter 4

System Architecture and Design

Chapter 4: System Architecture and Design

4.1 Overview

This chapter lays out how the system is structured from the frontend and backend layers down to the database tables and includes the diagrams used to document the design. Rather than repeating the methodology details from the previous chapter, the focus here is on the concrete architectural decisions and the resulting data model.

4.2 Frontend Architecture

The client side is built around React's component model, with a few key technologies holding it together:

- React components handle all UI rendering and user interaction.
- A Redux store provides one centralized place for application-wide state.
- Tailwind CSS takes care of styling and responsive behavior.
- TypeScript adds static type checking, which helps catch mistakes before they reach the browser.

4.3 Backend Architecture

The server-side API is written in .NET Core and sticks to RESTful design principles [7]. Its internal structure keeps different responsibilities cleanly separated:

- **Controllers:** Accept HTTP requests, validate inputs, and return properly formatted responses. Each feature module has a dedicated controller.
- **Services:** Where the business rules actually live. Domain operations, cross-module coordination, and anything requiring non-trivial logic are handled at this layer.

- **Data Access:** Entity Framework Core, via the `AppDbContext` class, manages database queries and persistence.
- **Middleware:** Handles concerns that cut across the whole application: authentication checks, authorization enforcement, CORS policies, and serving uploaded files.
- **DTOs:** Request and response shapes are defined as separate data contracts, so internal schema changes do not automatically break the external API surface.

4.4 Database Schema and Data Dictionary

All persistent data lives in a SQL Server database, with Entity Framework Core managing the schema through code-first migrations [9]. The tables below are representative of the core data model.

4.4.1 Table Campaigns

This table drives the entire fundraising lifecycle covering creation, progress tracking, and eventual completion or cancellation.

Table 4.1: Data Dictionary - Campaigns Entity

Column Name	Data Type	Constraint	Description
Id	INT	PRIMARY KEY	Auto-increment campaign identifier.
Title	NVARCHAR(200)	NOT NULL	Campaign title displayed to donors.
TargetAmount	DECIMAL(18,2)	NOT NULL	Goal numerical value.
AmountRaised	DECIMAL(18,2)	DEFAULT(0.0)	Cumulatively dynamically updated value.
Status	NVARCHAR(50)	NOT NULL	State such as pending, active, completed, cancelled.
CreatedBy	INT	FOREIGN KEY	User who created the campaign.
CreatedAt	DATETIME2	DEFAULT(GETDATE())	Record creation timestamp.

4.4.2 Table Donations

Each row here records a single donation transaction, linking a donor to a specific campaign.

Table 4.2: Data Dictionary - Donations Entity

Column Name	Data Type	Constraint	Description
Id	INT	PRIMARY KEY	Donation record identifier.
CampaignId	INT	FOREIGN KEY	Parent campaign reference.
UserId	INT (nullable)	FOREIGN KEY	Registered donor reference (nullable for guest donations).
Amount	DECIMAL(18,2)	NOT NULL	Donation amount.
PaymentReference	NVARCHAR(MAX)	NULLABLE	Gateway transaction reference value.
Status	NVARCHAR(50)	NOT NULL	pending, completed, failed, or refunded.

4.4.3 Table Volunteer Reports

Volunteers can file reports about misconduct, no-shows, or other issues against fellow volunteers. This table stores those reports along with their review status.

Table 4.3: Data Dictionary - Volunteer Reports

Column Name	Data Type	Constraint	Description
Id	INT	PRIMARY KEY	Report identifier.
ReportedByVolunteerId	INT	FOREIGN KEY	Volunteer profile that submitted the report.
ReportedVolunteerId	INT	FOREIGN KEY	Volunteer profile being reported.
ReportType	NVARCHAR(MAX)	NOT NULL	Type such as misconduct, no-show, or other.
Description	NVARCHAR(MAX)	NOT NULL	Detailed narrative of the incident.
Status	NVARCHAR(MAX)	NOT NULL	pending, under_review, resolved, or rejected.

Beyond these three core tables, the database includes several other entities that round out the platform:

- **Users:** Stores identity data, assigned roles, and profile information.
- **Testimonials:** Holds donor feedback along with computed sentiment scores and moderation flags.
- **CampaignUpdates:** Contains progress posts and milestone announcements published by campaign creators.
- **PhysicalDonations:** Tracks in-kind contributions collected by volunteers, with OTP verification records.

- **Vouchers, ReserveFunds, Withdrawals:** Together, these tables manage expense approval workflows, reserve fund balances, and controlled disbursements.

4.5 ER Diagram

The ER diagram below maps out the relationships between entities with one-to-many links between Users and Donations, Campaigns and Donations, and so forth.

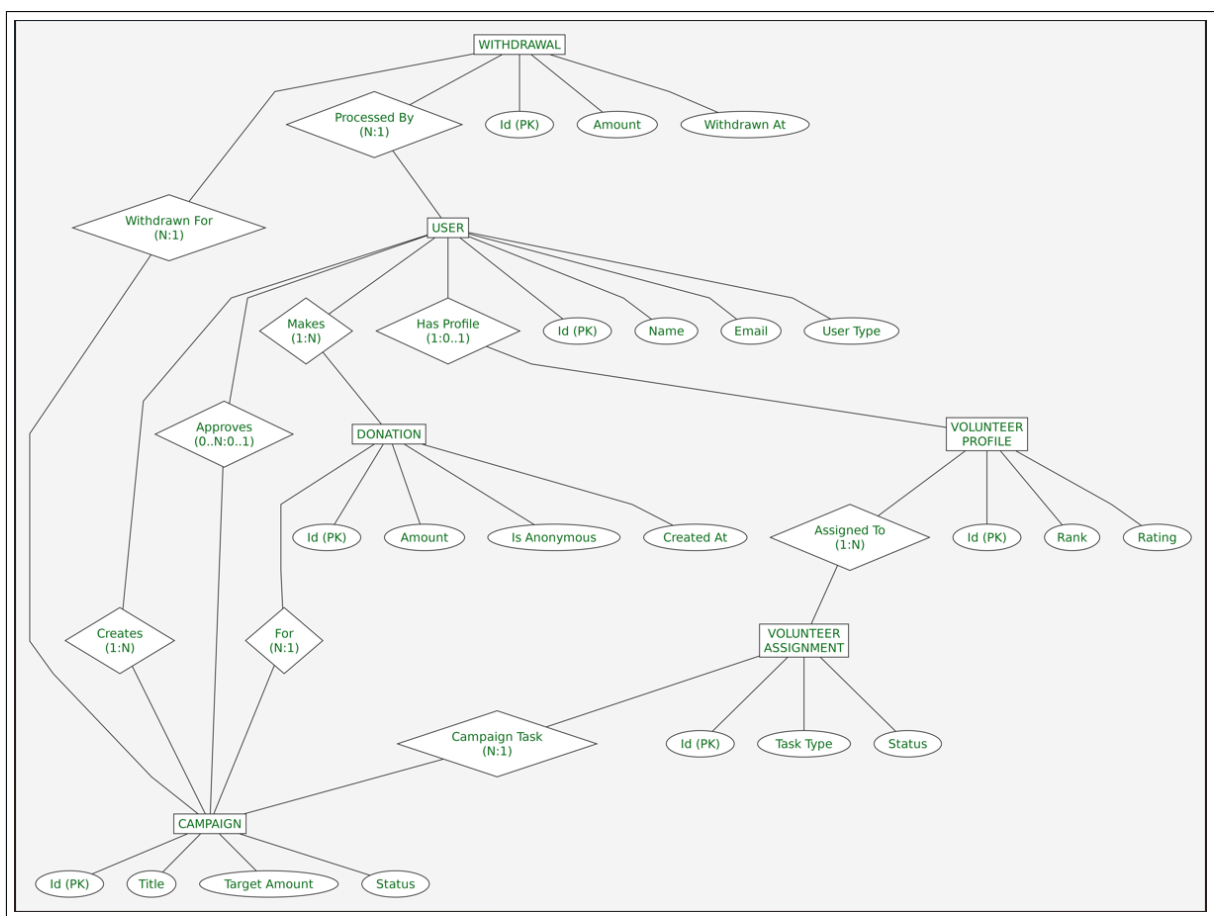


Figure 4.1: Entity-Relationship Diagram.

4.6 Use Case Diagram

The use case diagram below shows which actions are available to each role. Admins have the broadest access, while Donors, Volunteers, and Campaign Creators each see a focused subset

of the system:

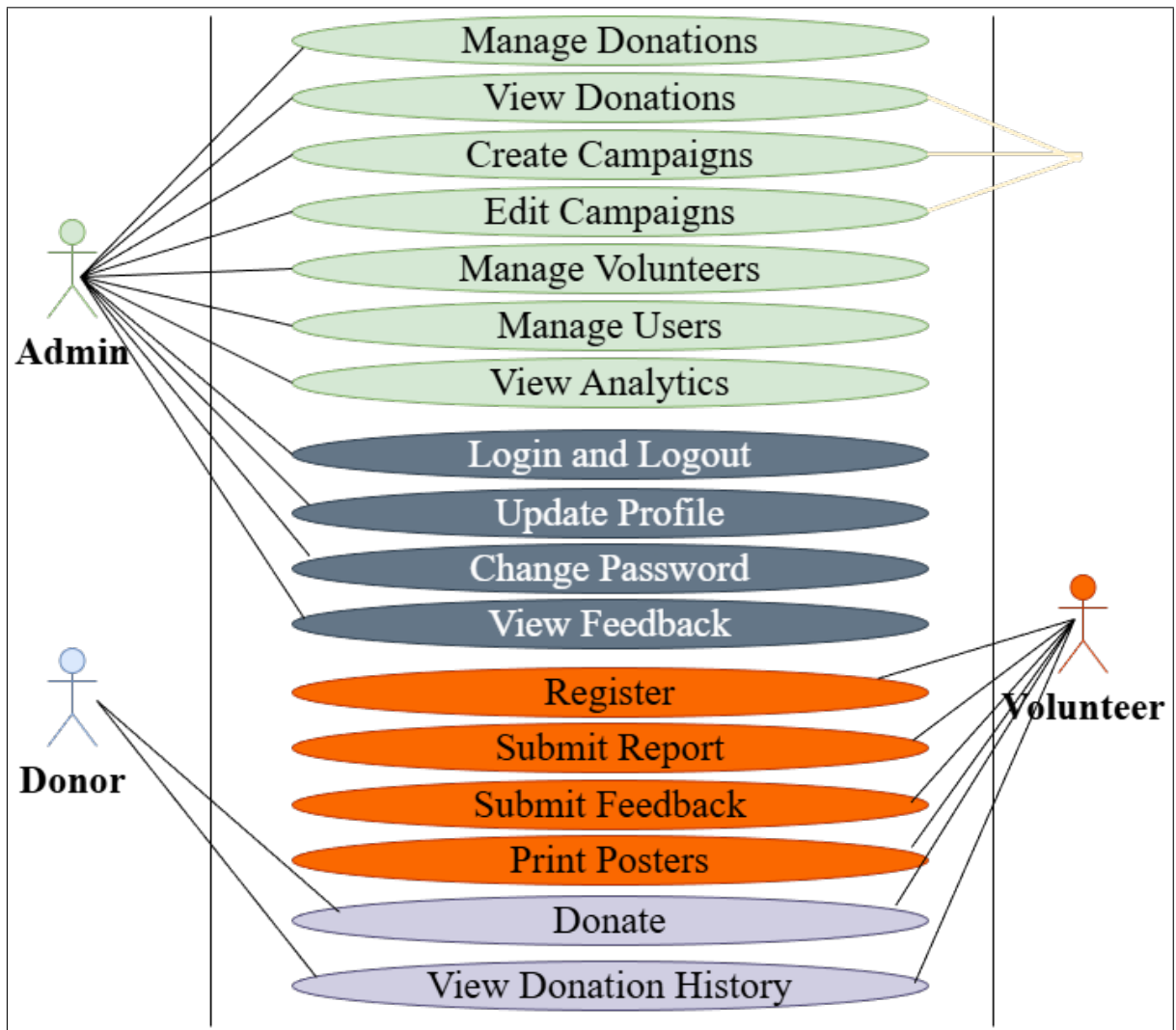


Figure 4.2: Use Case Diagram.

4.7 Schema Diagram

This diagram provides a closer look at every table in the database, including column names, data types, and the foreign key connections between them:

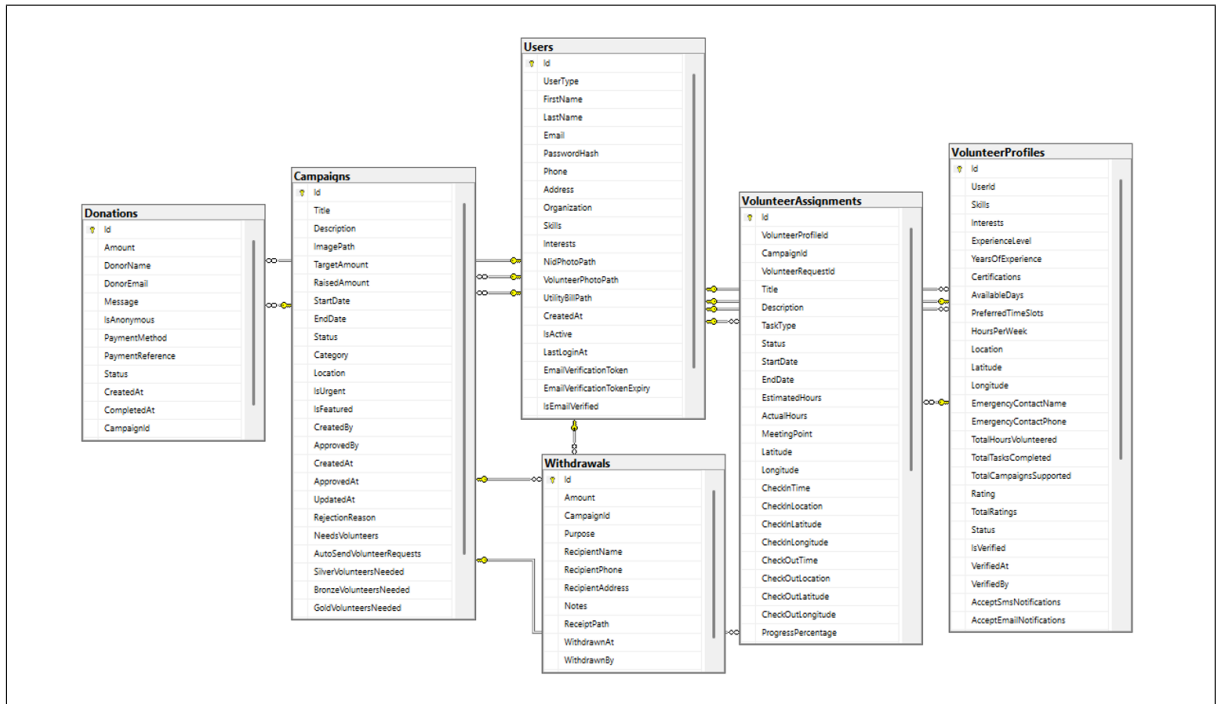


Figure 4.3: Database Schema Diagram.

4.8 System Context Diagram

At the highest level of the C4 model (Level 1), we can see the DMS interacting with external actors and external systems like SSLCommerz and email/SMS providers.

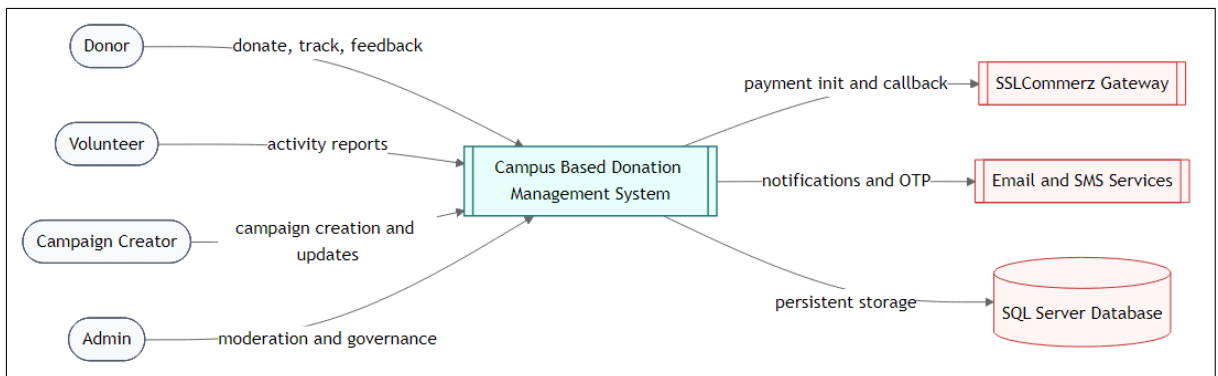


Figure 4.4: System Context Diagram.

4.9 Container Diagram

Going one level deeper (C4 Level 2), the container diagram breaks the system into its major runtime pieces: the React SPA, the .NET API, the SQL Server database, and the third-party services it depends on.

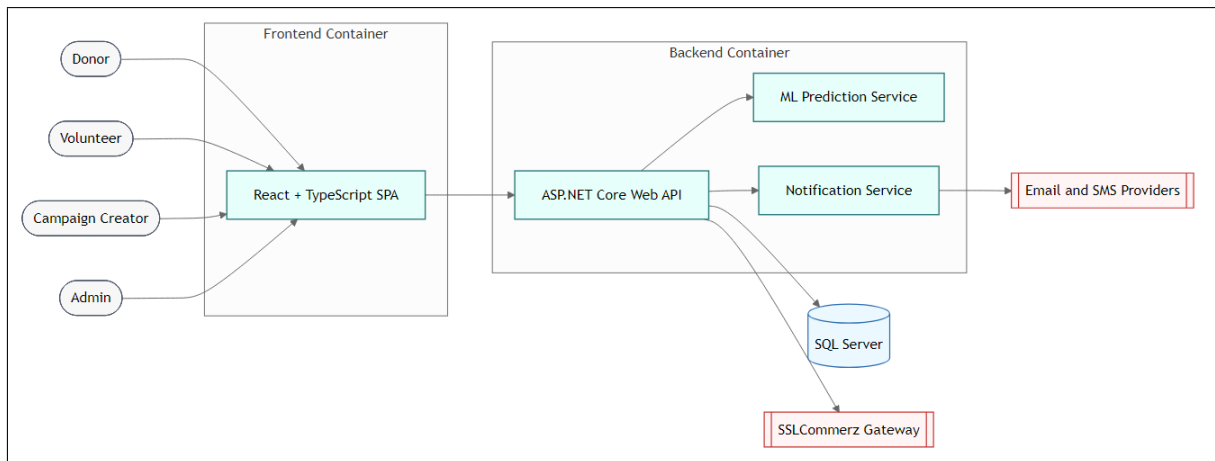


Figure 4.5: Container Diagram.

4.10 Component Diagram

Inside the backend container, the component diagram shows how controllers, services, and data-access classes connect to one another.

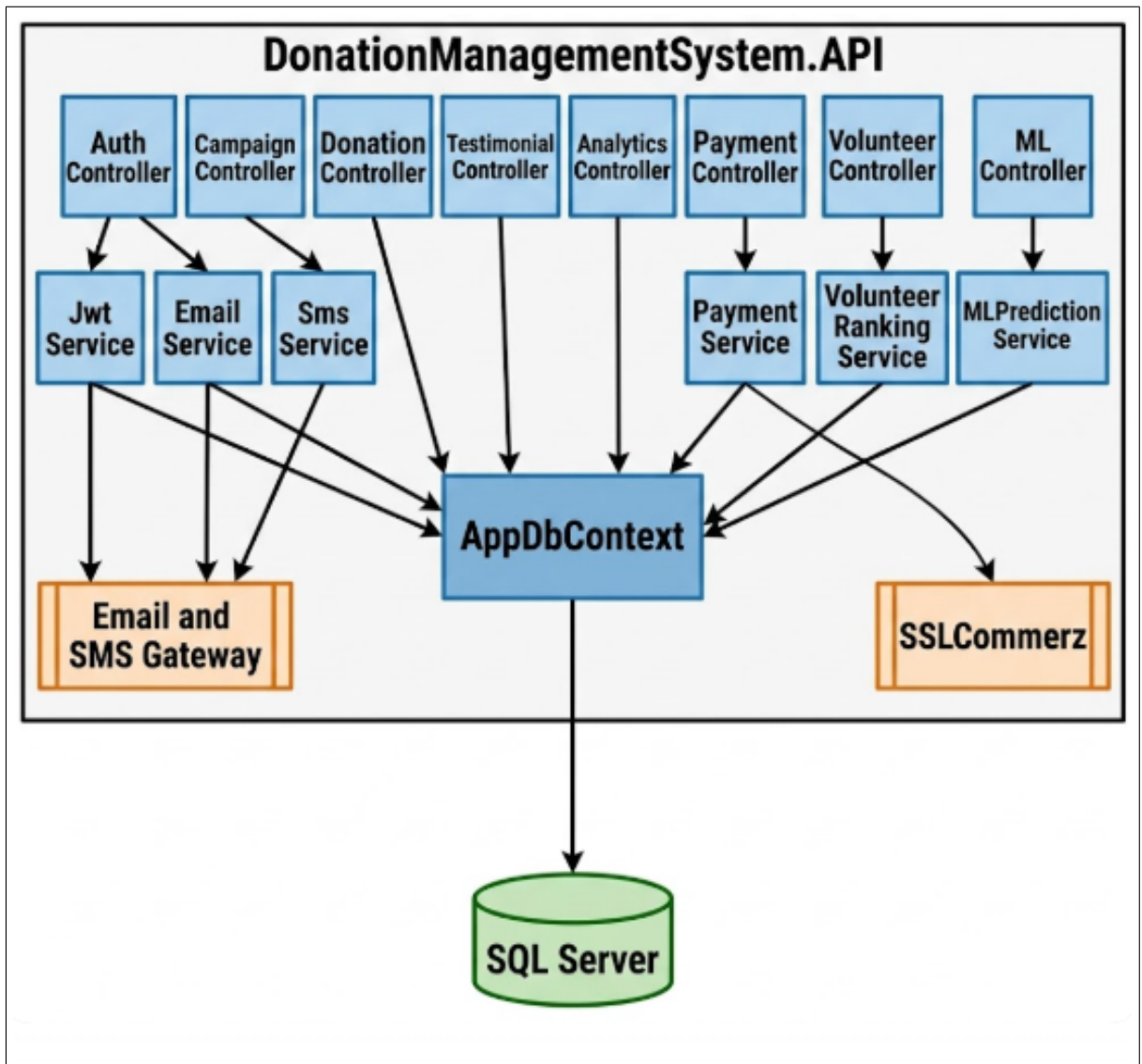


Figure 4.6: Component Diagram.

4.11 Class Diagram

A simplified class diagram captures the main domain objects such as Users, Campaigns, and Donations and the relationships between them.

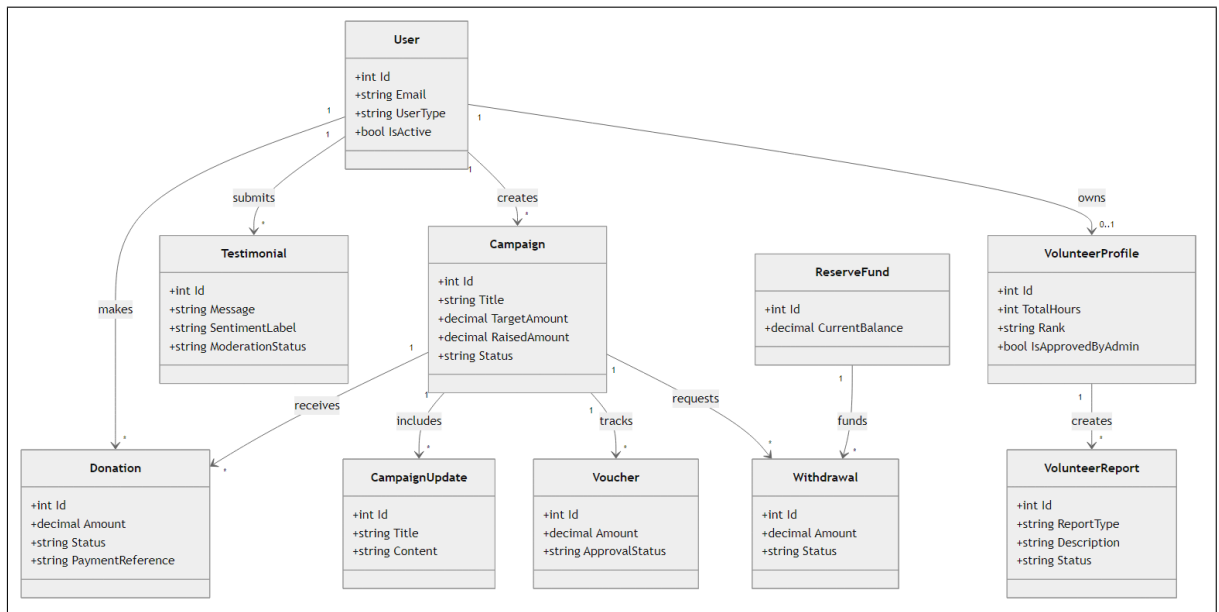


Figure 4.7: Class Diagram.

4.12 Data Flow Diagram (Level 0)

The Level 0 DFD draws a boundary around the entire system, showing external entities (donors, admins, payment gateway) and the high-level data stores they interact with.

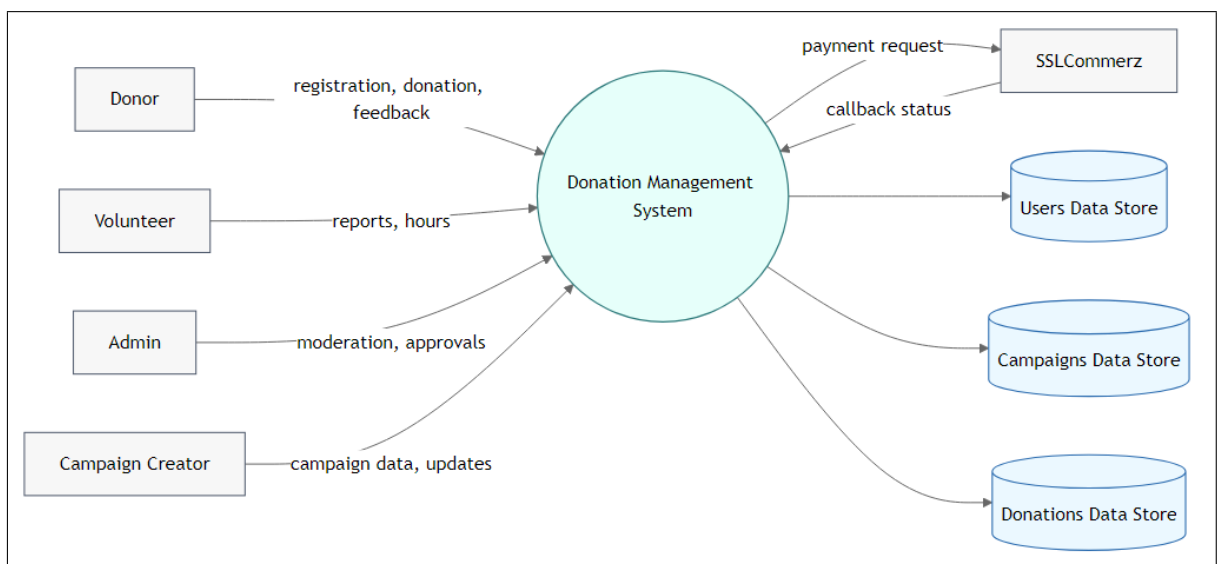


Figure 4.8: Data Flow Diagram (Level 0).

4.13 Data Flow Diagram (Level 1)

Drilling into Level 1, the major process is split into module-level groups like authentication, donations, campaigns, and analytics while the data movements between them become visible.

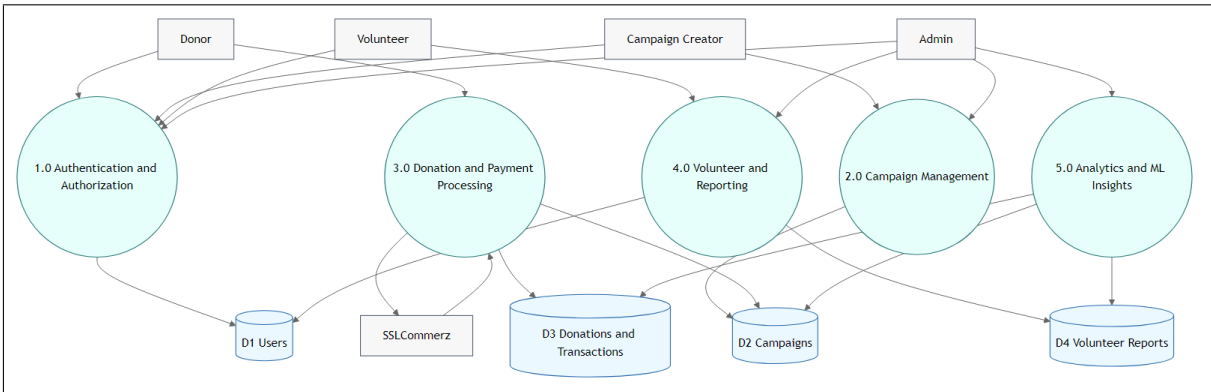


Figure 4.9: Data Flow Diagram (Level 1).

4.14 Deployment Diagram

The deployment diagram maps each software container to the infrastructure node that hosts it, along with connections to external integrations.

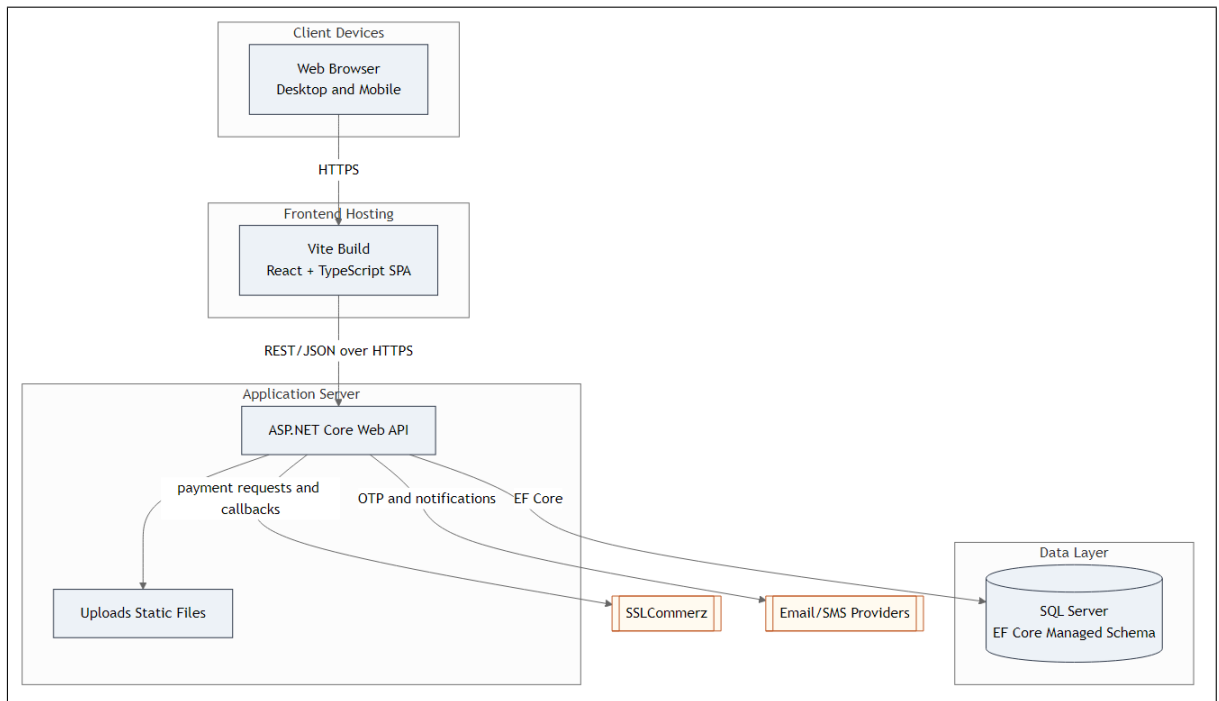


Figure 4.10: Deployment Diagram.

4.15 Activity Diagram: Donation Workflow

This activity diagram traces what happens from the moment a donor clicks “Donate” to the final status update covering payment initiation, gateway redirect, callback processing, and record finalization.

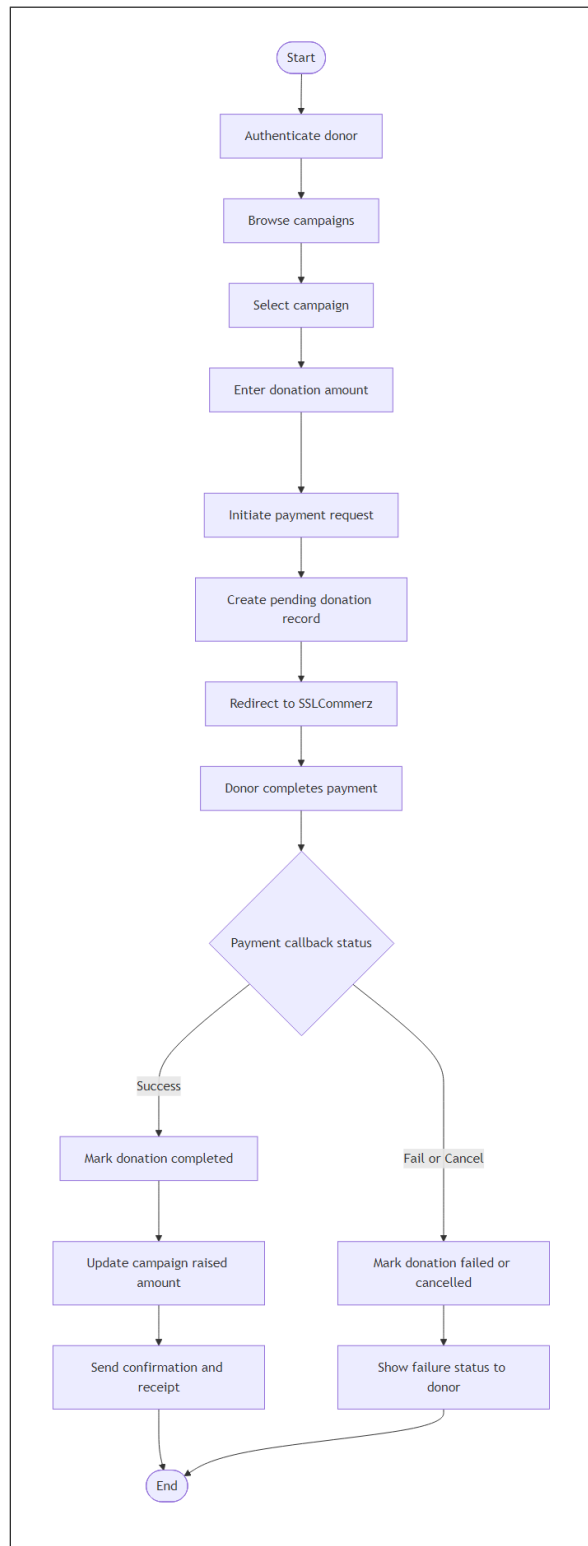


Figure 4.11: Activity Diagram for Donation Workflow.

4.16 Campaign Lifecycle State Diagram

Campaigns move through a defined set of states such as pending approval, active, completed, and cancelled with only certain transitions allowed. The diagram below formalizes those rules.

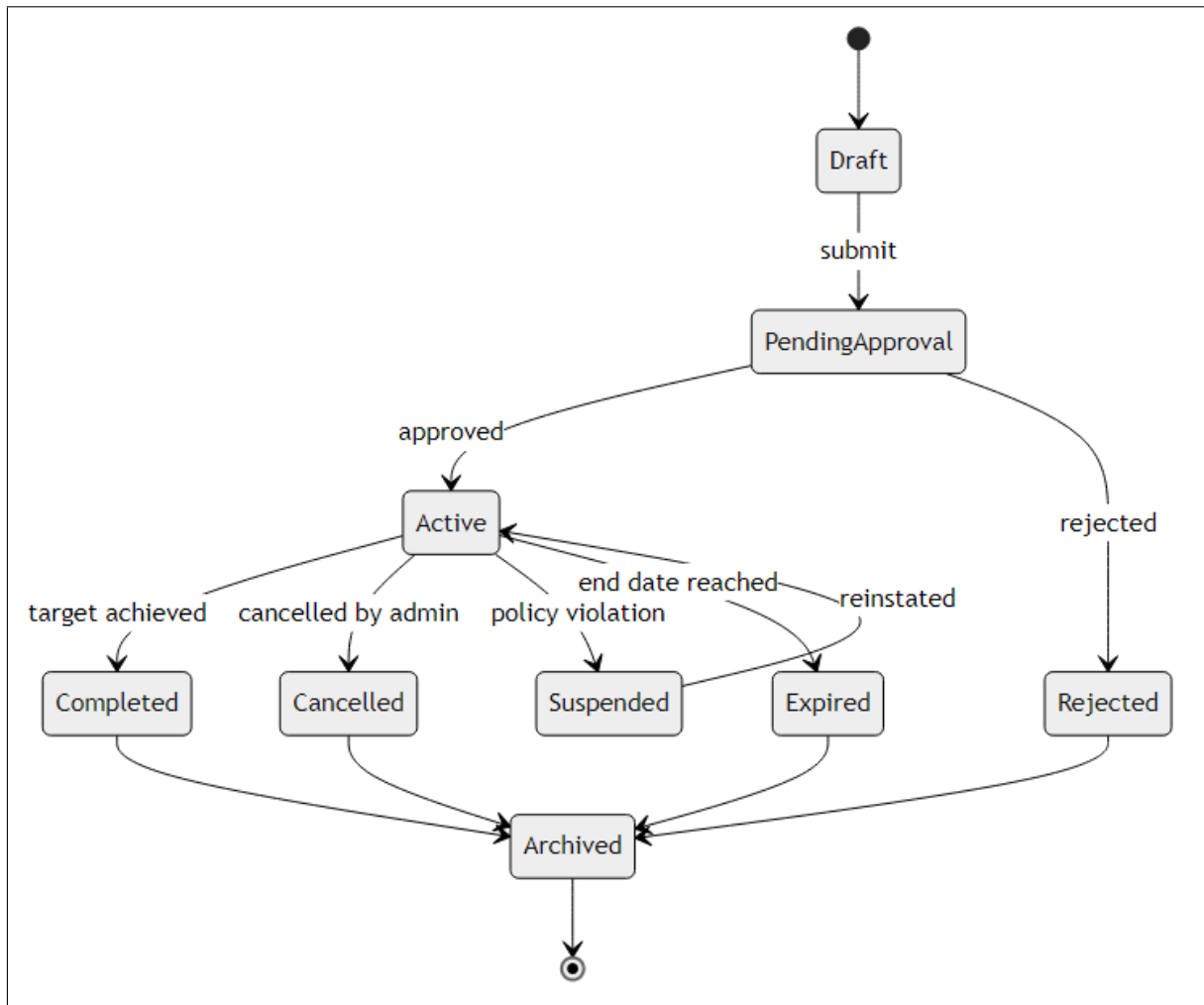


Figure 4.12: Campaign Lifecycle State Diagram.

4.17 Authentication Flow

Authentication relies on JWTs with role-aware authorization [25]. Here is how the process works in practice.

4.17.1 Registration, Login, and Token Issuance

1. When a new user registers, their password is hashed with BCrypt and stored never in plain text [26].
2. An email verification token is generated and sent to the user's inbox.
3. At login, the server verifies credentials and returns both a JWT access token and a refresh token.
4. When the access token expires, the refresh endpoint checks the refresh token's validity and issues a fresh access token without requiring the user to log in again.

4.17.2 Authorization and Access Control

Once a user is authenticated, every request to a protected endpoint goes through additional checks:

- ASP.NET Core's JWT bearer middleware validates the token's signature, issuer, audience, and expiry.
- Claims embedded in the token are inspected to enforce role boundaries so only admin-role tokens can reach user-management endpoints [10].
- Profile updates and other user-specific actions require that the token's identity claim matches the target user.

4.18 Payment Processing and Ledger Integration

All monetary transactions flow through the SSLCommerz gateway, with the backend handling post-payment synchronization via callbacks [3].

4.18.1 Sequence Diagram: Payment Callback Flow

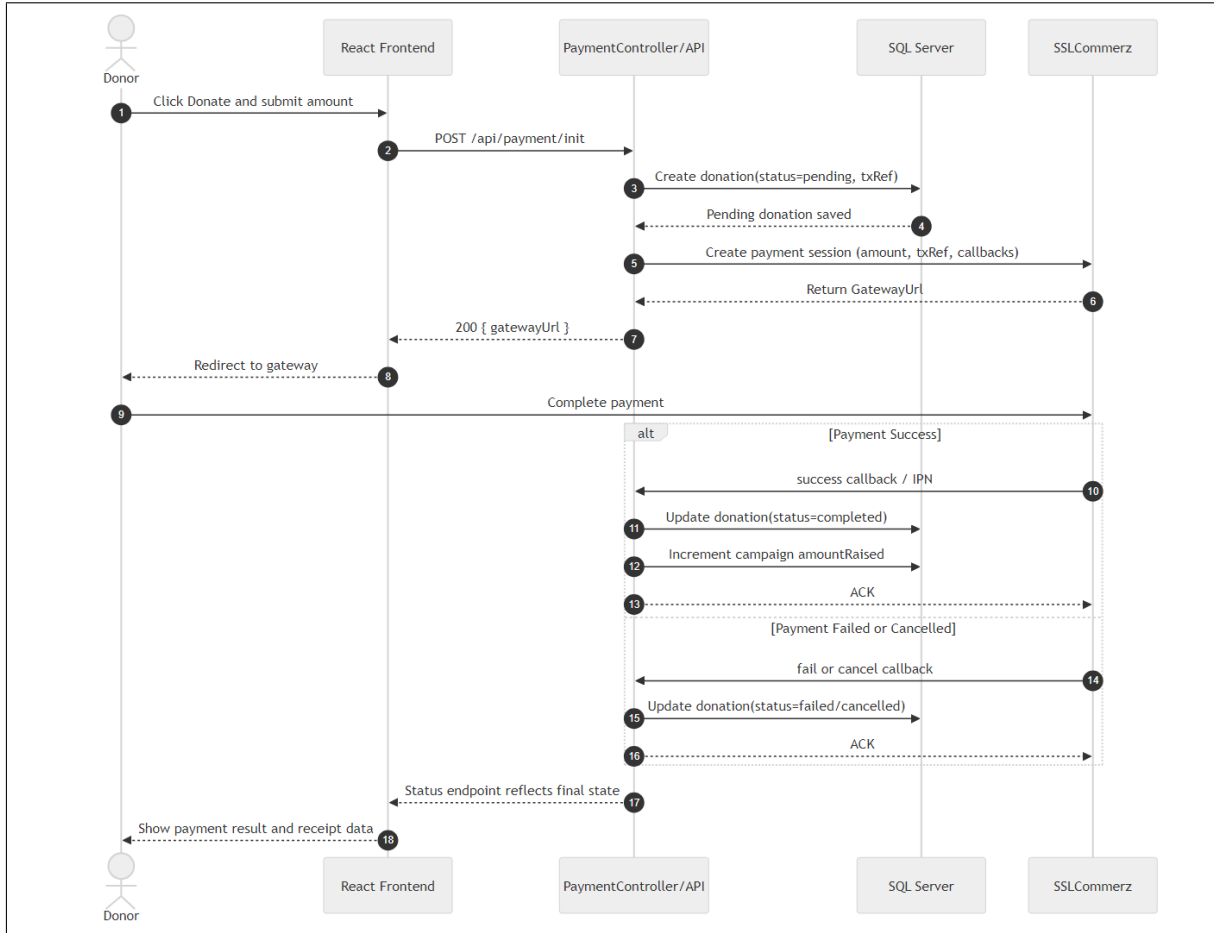


Figure 4.13: Sequence Diagram for Donation Payment Flow.

4.18.2 How a Transaction Starts

1. The donor clicks the payment button on the frontend, sending a request to the backend.
2. PaymentController creates a donation record in “pending” status and generates a unique transaction reference.
3. Payment metadata including amount, donor details, and callback URLs is sent to the SSLCommerz API.
4. The user’s browser is redirected to the gateway page returned by SSLCommerz.

4.18.3 Callback Handling and Idempotency

After the donor completes (or abandons) payment, SSLCommerz sends callbacks back to the server:

- The callback payload is matched to the corresponding donation via the donation ID or transaction reference.
- If payment succeeded, the donation's status moves from "pending" to "completed."
- The campaign's raised amount is incremented accordingly; any overflow beyond the target can be redirected into a reserve fund entry.
- Idempotency checks ensure that processing the same callback twice does not result in duplicate balance updates.

Chapter 5

System Implementation

Chapter 5: System Implementation

5.1 System Setup

Getting the DMS running involves two main pieces: a React-based frontend served as static files, and an ASP.NET Core backend that exposes the API and connects to the database. This section covers the technology choices and then walks through the implemented features with screenshots.

5.1.1 Technologies Used

- **Frontend Technologies:**

Table 5.1: Frontend Technologies.

SN	Technology
1	React
2	TypeScript
3	Redux Toolkit
4	Tailwind CSS
5	Axios (HTTP Client)
6	Vite (Build Tool)

- **Backend Technologies:**

Table 5.2: Backend Technologies.

SN	Technology
1	.NET Core 7/8
2	C#
3	Entity Framework Core
4	SQL Server
5	ASP.NET Core Identity
6	JWT Authentication

5.2 System Implementation

The screenshots below follow the same sequence a real user would encounter: logging in, navigating the admin dashboard, managing campaigns, making a donation, and so on. This ordering makes it easier to see how different parts of the system connect during actual use.

5.2.1 User Authentication

The login screen is the entry point. Behind it, JWT-based sessions handle identity, email verification ensures genuine accounts, and refresh tokens keep users logged in without repeated password prompts.

5.2.3 User and Role Management

Admins can assign roles, enable or disable accounts, and manage user profiles from a dedicated panel.

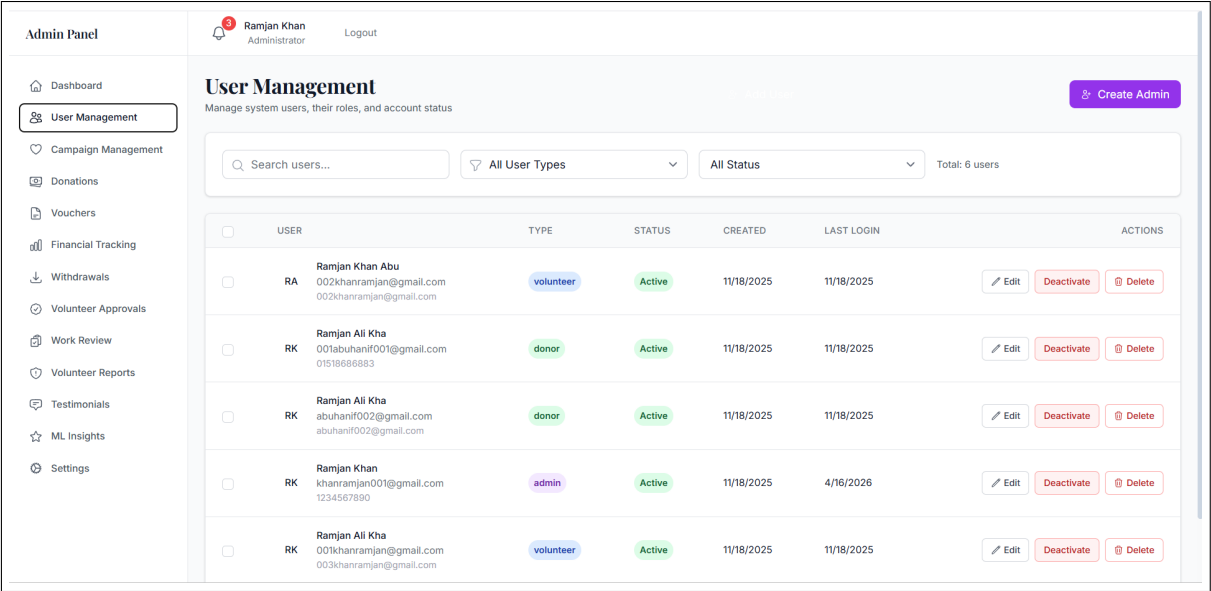


Figure 5.3: User and Role Management Panel.

5.2.4 Campaign Management

Campaign creators use this interface to set up new campaigns, define fundraising targets, upload images, post progress updates, and track how close they are to their goal.

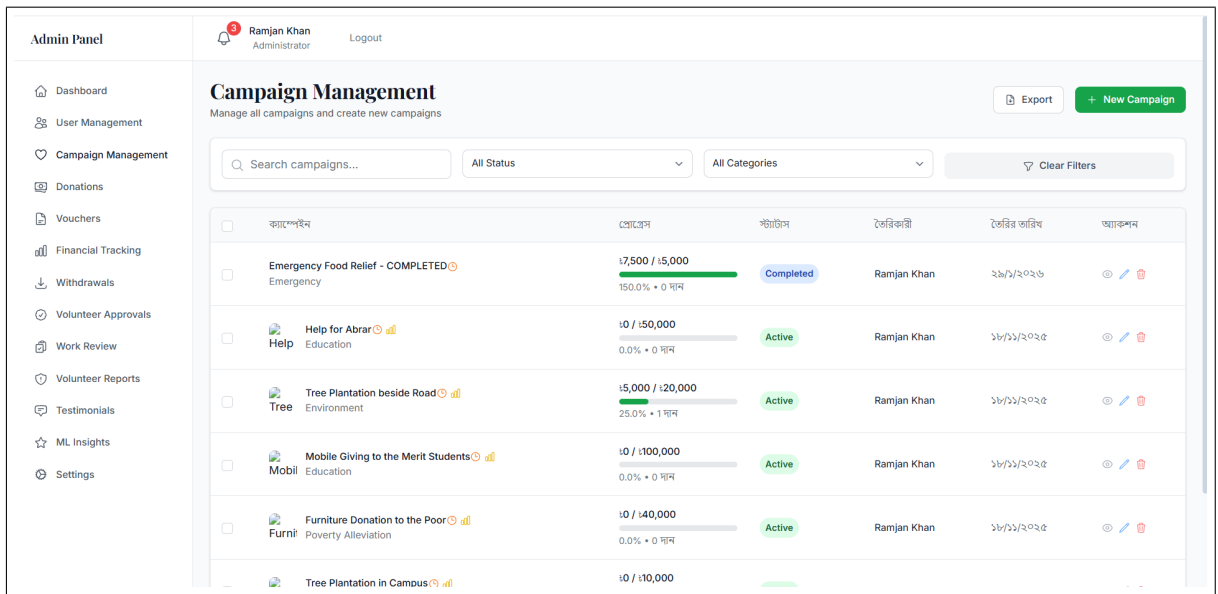


Figure 5.4: Campaign Creation and Management Interface.

5.2.5 Donation Processing

Donors go through a checkout flow that connects to SSLCommerz for payment. Once the transaction is confirmed, the system updates the donation status and campaign totals automatically.

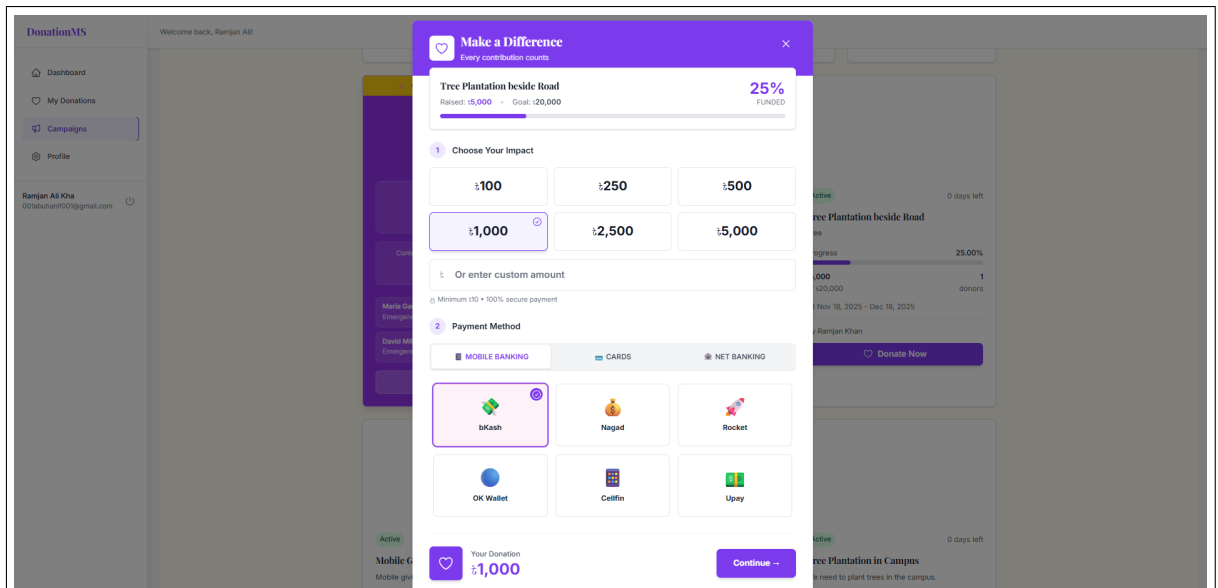


Figure 5.5: Donation Checkout and Payment Initiation.

5.2.6 Volunteer Management

Volunteers have their own panel where they can submit activity reports, log hours, see their overall progress, and check how their contributions stack up.

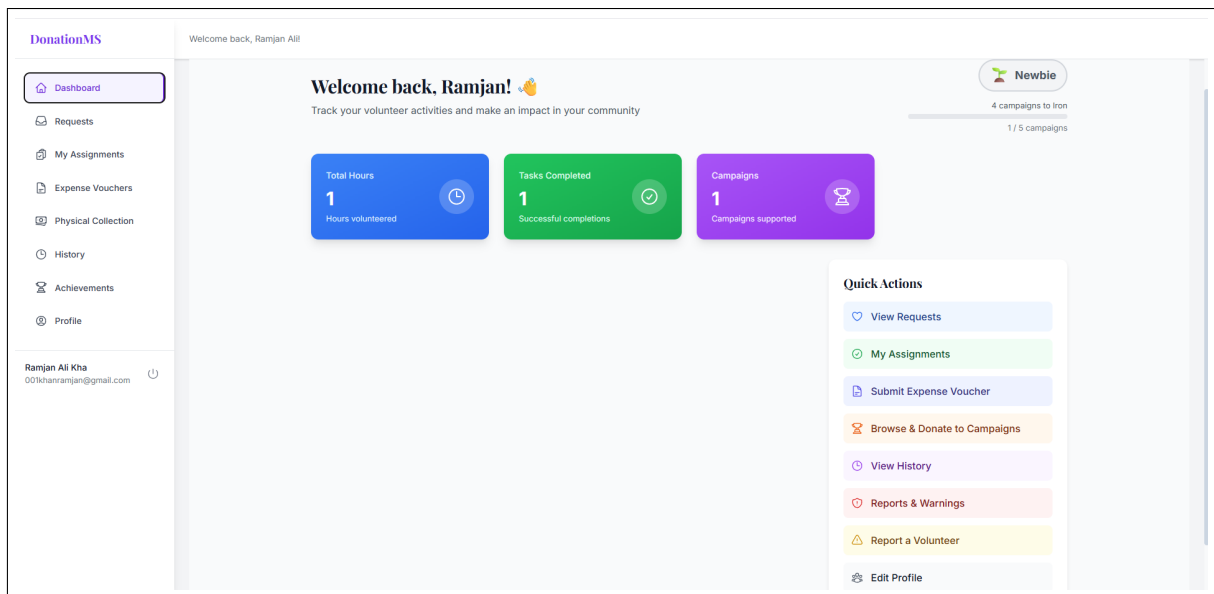


Figure 5.6: Volunteer Activity and Reporting Interface.

5.2.7 Testimonial and Sentiment Analysis

When donors leave testimonials, the system scores each one for sentiment. Negative-scoring entries are held for admin review before appearing publicly.

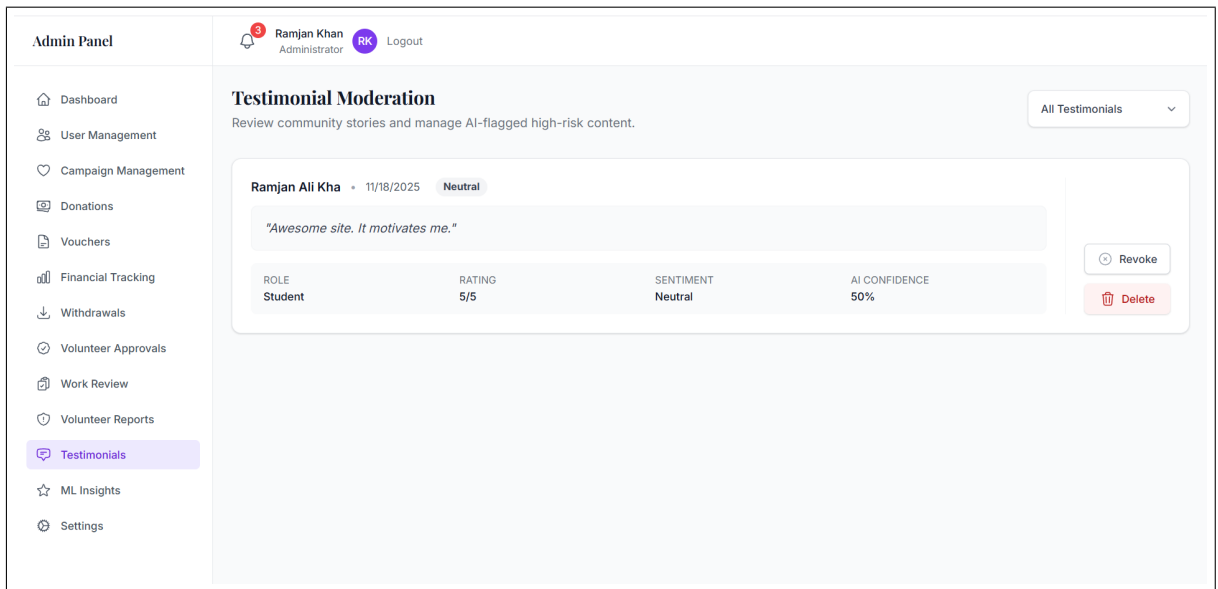


Figure 5.7: Testimonial Moderation and Sentiment Review.

5.2.8 Analytics and Reporting

The analytics dashboard pulls together donation trends, campaign performance, volunteer output, and donor activity into interactive charts and summary cards.

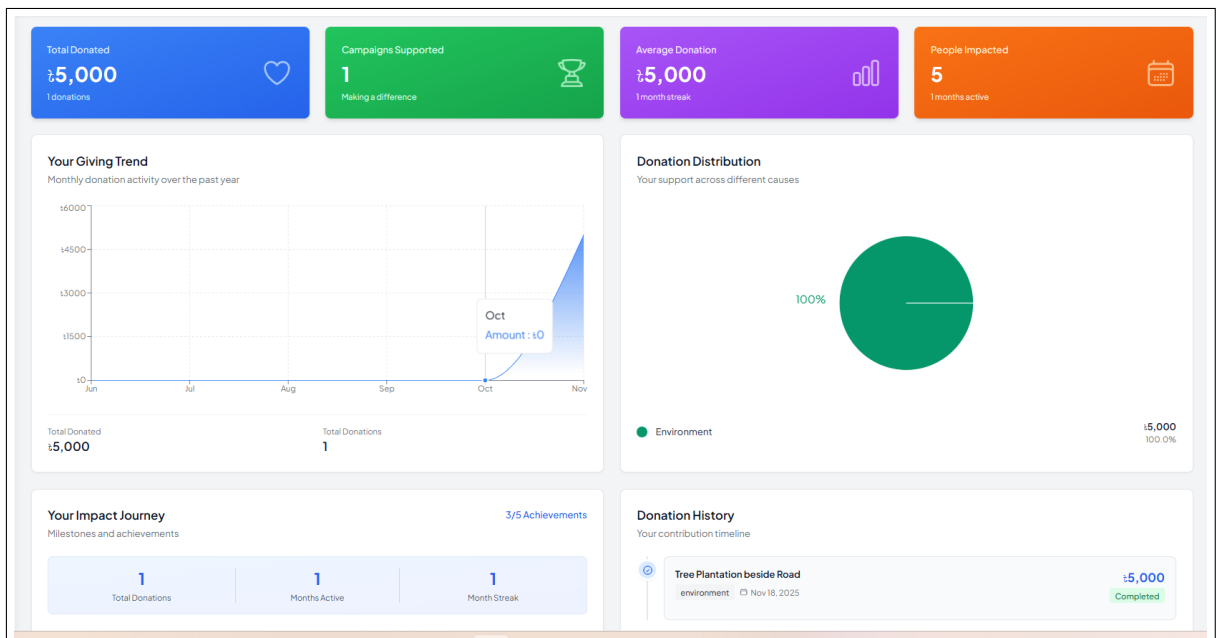


Figure 5.8: Analytics and Reporting Dashboard.

5.2.9 Additional Feature Screenshots

The remaining screenshots cover the other major modules built into the platform:

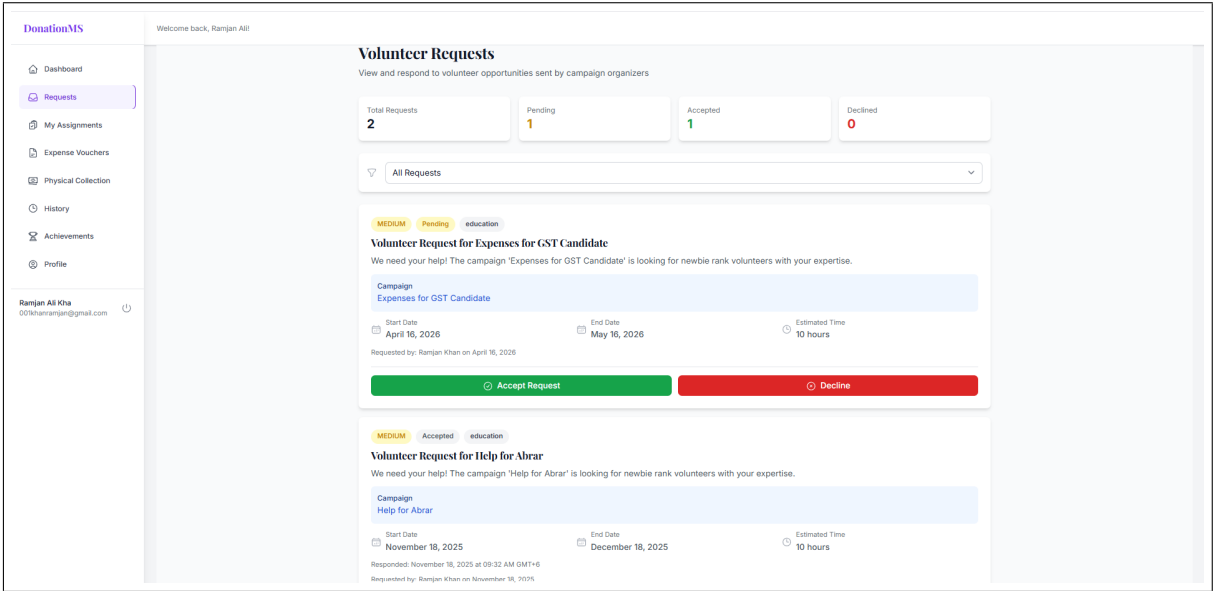


Figure 5.9: Campaign Updates and Announcement Interface.

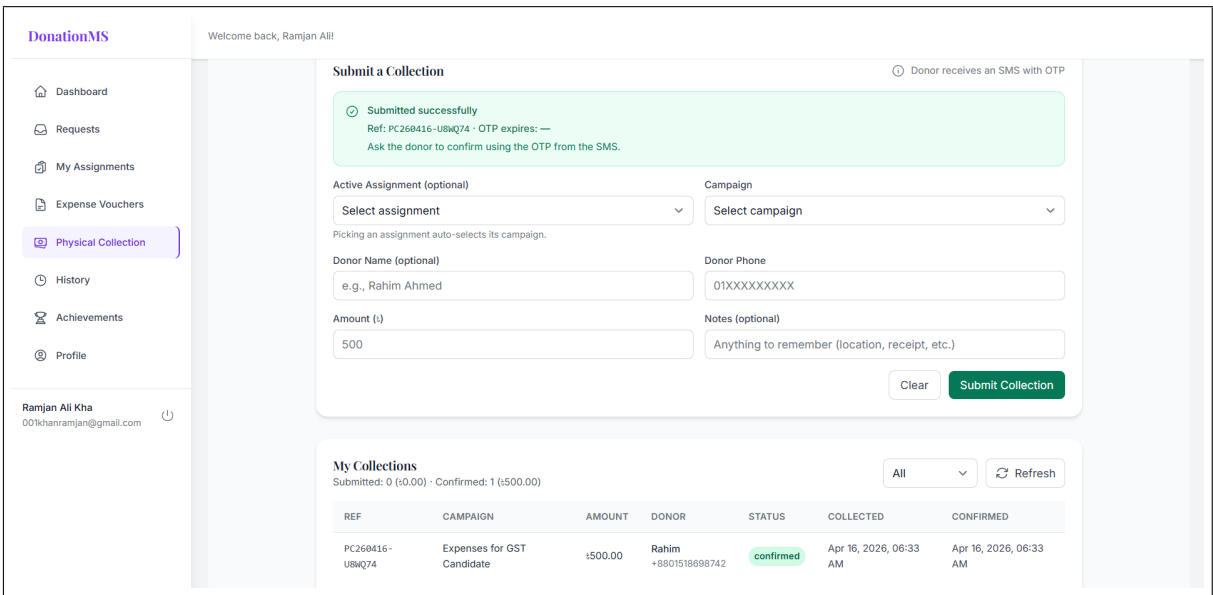


Figure 5.10: Physical Donation Verification and Tracking.

Submit Expense Voucher

Submit your expense voucher for relief distribution work. Only completed campaigns are eligible.

Campaign *

Select a completed campaign

Only completed campaigns are shown

Expense Category *

Select a category

Expense Date *

04/16/2026

Description *

Describe what the expense was for and why it was necessary

Provide context about the expenses and their purpose

Expense Items

+ Add Item

Item Name *

e.g., Rice bags

Purchase Date *

04/16/2026

Unit Price *

0

Quantity *

1

Notes

Additional notes about this item

Subtotal: \$0.00

Total Amount:

\$0.00

Receipt / Proof (Optional)

Choose File

No file chosen

Upload a photo or PDF of your receipts (Max: 5MB, JPG/PNG/PDF)

Cancel

Submit Voucher

Figure 5.11: Expense Voucher Management and Approval Workflow.

Page 61 of 82

Emergency Food Relief – COM...

Raised

Completed Withdrawn

Pending + Approved

Available

Tree Plantation beside Road

Raised

Completed Withdrawn

Pending + Approved

Available

Winter Cloth

Raised

Completed Withdrawn

Pending + Approved

Available

Expenses for GST Candi

Raised

Completed Withdrawn

Pending + Approved

Available

Tree Plantation in Campus

Raised

Completed Withdrawn

Pending + Approved

Available

Furniture Donation to th

Raised

Completed Withdrawn

Pending + Approved

Available

All Campaigns

All s

REFERENCE	DATE						
WD-20260416-8D2DBB	4/16/2026, 9:06:39 AM	Emergency Food Relief – COMPLETED	₹10	pending	Ramjan Khan	Cancel	Own request: awaiting another admin Details
WD-20260416-968105	4/16/2026, 8:49:01 AM	Expenses for GST Candidate	₹500	pending	Ramjan Khan	Cancel	Own request: awaiting another admin Details

Create Withdrawal Request

Step 1 of 2: Fill details before confirmation.

Campaign

Select campaign

Amount (BDT)

e.g. 2500.00

Amount is required

Purpose

Purpose of fund release

Recipient Name

Optional

Recipient Phone

Optional

Recipient Address

Optional

Notes

Optional internal notes

Cancel

Preview Request

Figure 5.12: Withdrawal Request and Approval Interface.

Financial Dashboard

Track all donations and campaign finances

Total in System

₹27,000

Campaign Funds

₹19,000

Reserve Fund

₹8,000

Active Campaigns

7

Completed

1

Campaign Breakdown

Reserve Fund

Reserve Fund Entries

Overflow donations from completed campaigns

DATE	CAMPAIGN	DONOR	DESCRIPTION	AMOUNT
3/5/2026	Emergency Food Relief - COMPLETED	Maria Garcia	Overflow from Medical Emergency campaign - Helping community reserve	₹3,000
2/28/2026	Emergency Food Relief - COMPLETED	David Miller	Overflow donation - Campaign exceeded goal, extra funds for future emergencies	₹2,500
2/23/2026	Emergency Food Relief - COMPLETED	Anonymous Donor	Generous overflow contribution to community reserve fund	₹1,500
2/18/2026	Emergency Food Relief - COMPLETED	Sophia Williams	Overflow from Emergency Food Relief campaign (donated ₹3000 when ₹2000 was needed)	₹1,000

Total Reserve

₹8,000

Figure 5.13: Reserve Fund Transparency.

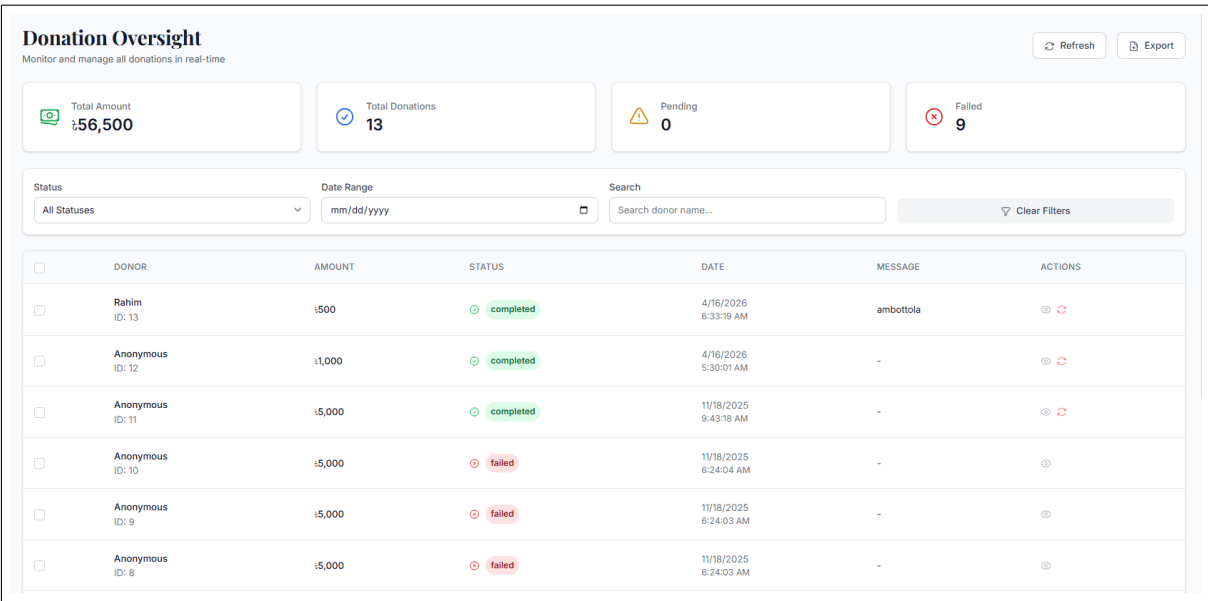


Figure 5.14: Payment Transaction History and Status Tracking.

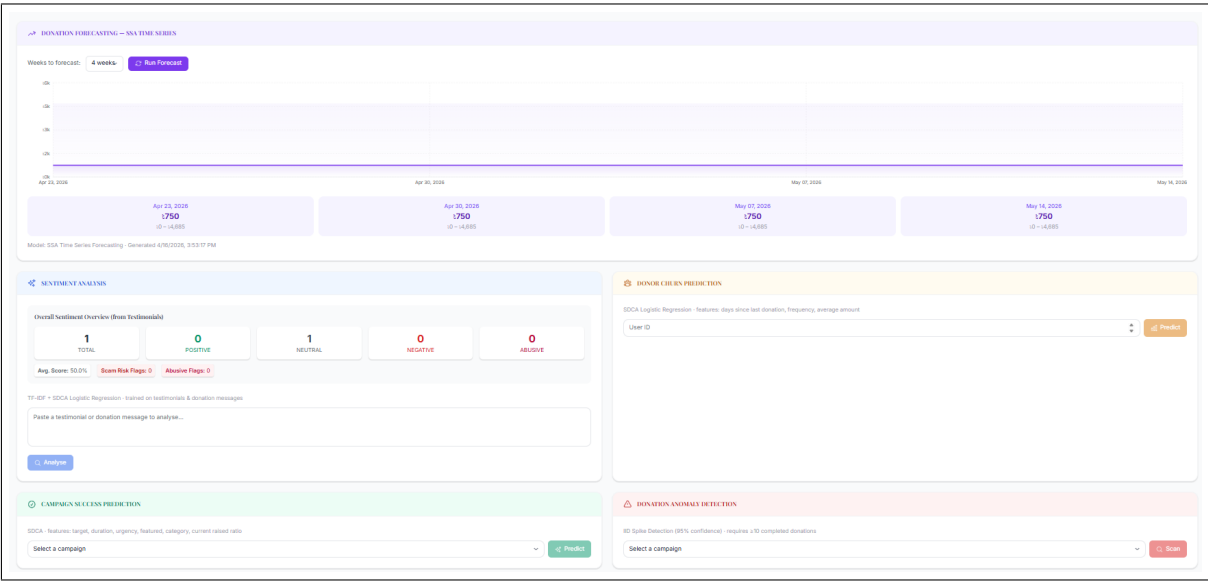


Figure 5.15: Machine Learning Insights and Prediction Outputs.

5.2.10 Screenshot File Arrangement

For consistency and maintainability, implementation screenshots should be stored under Report/images using the following filenames:

Table 5.3: Implementation Screenshot Naming Scheme

Filename	Purpose
login_page.png	Authentication/login screen.
admin_dashboard.png	Main administrative overview and metrics.
user_management.png	User, role, and access control management.
campaign_management.png	Campaign create/update and progress monitoring.
donation_checkout.png	Donation amount entry and payment initiation flow.
volunteer_panel.png	Volunteer reporting and activity tracking.
testimonial_moderation.png	Sentiment-based moderation and feedback review.
analytics_dashboard.png	Reports and analytical visualizations.
campaign_updates.png	Campaign announcement and progress update page.
physical_donation.png	Physical donation collection and OTP verification workflow.
voucher_management.png	Expense voucher submission and approval panels.
withdrawal_management.png	Withdrawal request queue and administrative decisions.
reserve_fund_dashboard.png	Public reserve fund summary and transaction visibility.
payment_transactions.png	Payment status log, callback result, and transaction history.
ml_predictions.png	Forecasting, anomaly, and churn prediction outputs.

5.3 Key Features Implementation

5.3.1 Donation Tracking and Processing

Donors and campaign creators can both see, in real time, how much has been raised against the target. Each individual donation is recorded with a timestamp, amount, and payment status. The processing module hooks into SSLCommerz so that payments are confirmed and receipts generated without manual intervention.

5.3.2 Campaign Updates and Communications

Campaign creators post progress updates including stories about how the funds are being used, milestone announcements, or thank-you notes. These updates keep donors informed and engaged even after they have contributed.

5.3.3 Volunteer Reporting and Recognition

Volunteers log their activities by submitting reports that include hours worked, a description of what they did, which campaign they were supporting, and any particular achievements. The system uses this data for merit-based ranking and public recognition.

5.3.4 Physical Donation Management

In-kind donations including clothes, food, and supplies are handled through a dedicated tracking module:

- OTP-based donor verification via SMS.
- Structured item documentation and valuation.
- Integration with donation ledger systems.
- Audit trail generation for institutional accountability.

5.3.5 Expense Voucher Management

Campaign-related expenses go through a structured voucher process:

- Structured voucher creation with detailed expense documentation.
- Hierarchical approval workflows.
- Supporting document attachment and verification.
- Real-time status monitoring and notifications.

5.3.6 Financial Controls and Withdrawal Management

Money does not leave the system without oversight:

- Formal withdrawal request workflows.
- Administrative approval requirements.
- Public reserve fund transparency dashboard.
- Automated notification infrastructure.

5.3.7 Payment Integration

The SSLCommerz integration supports credit cards, mobile banking (bKash, Nagad), and digital wallets. Transaction confirmations are handled automatically through server-side callbacks, so neither the donor nor the admin needs to do anything manually after payment.

5.3.8 Machine Learning Features

All ML capabilities run inside the backend as an in-process `MLPredictionService` built on ML.NET. The service is registered as a singleton, which means trained models stay in memory and get reused across requests rather than being retrained every time.

5.3.8.1 1. Donation Demand Forecasting (SSA)

Weekly donation totals are fed into ML.NET's Singular Spectrum Analysis pipeline to produce forecasts with confidence bounds [29, 31]. This gives campaign managers a rough idea of expected income over the coming weeks.

5.3.8.2 2. Donor Attrition Prediction

To identify donors who might stop contributing, the system builds a binary classification model using features like recency of last donation, total number of donations, average and total amounts, and donation frequency [30]. The output is a probability score plus a human-readable risk label (low, medium, high) to guide re-engagement efforts.

The probability estimate is interpreted as:

$$P(\text{Attrition}) = \sigma \left(\sum_{i=1}^n w_i f_i \right) \quad (5.1)$$

where σ denotes the sigmoid mapping to a value in $[0, 1]$.

5.3.8.3 3. Donation Anomaly Detection

ML.NET's IID spike detection algorithm scans completed donation streams for outliers [32].

The kinds of things it catches include:

- Single donations that are unusually large (or small) compared to the campaign's history.
- Sudden bursts of activity that do not match the campaign's normal pattern.

Any flagged transactions show up in the admin panel for manual review.

5.3.8.4 4. Sentiment Analysis

Testimonial text is featurized and classified using SDCA logistic regression. The model assigns each testimonial a sentiment band of positive, neutral, or negative based on confidence

thresholds. Negative-scoring entries automatically enter the moderation queue [22].

5.3.8.5 5. Campaign and Volunteer Intelligence

The service can also estimate a campaign's probability of reaching its goal, and it recommends suitable volunteers for assignment. Volunteer recommendations are scored by combining campaign attributes (category, urgency, remaining duration) with each volunteer's profile and past activity.

5.3.9 Email and SMS Notifications

The notification system handles several types of messages:

- Email-based donation receipt confirmations.
- Campaign update notifications.
- Administrative approval/rejection communications.
- SMS-based OTP delivery for donor verification.
- Real-time event-driven notifications.

5.3.10 Testimonial Moderation

As mentioned in the ML section, testimonials with negative sentiment are flagged automatically. Admins can approve, edit, or reject them before they appear on the public-facing campaign pages.

Chapter 6

System Evaluation and Results

Chapter 6: System Evaluation and Results

6.1 System Evaluation Overview

To check whether the system actually works as intended, we ran a combination of automated tests and manual scenario walkthroughs. The evaluation covered the main functional areas: authentication, donation workflows, volunteer features, testimonial moderation, and the analytics endpoints.

6.2 Automated Test Coverage

Both frontend and backend parts of the application have executable test suites. The table below summarizes what each layer validates.

Table 6.1: Implemented Evaluation Layers

Evaluation Layer	Primary Tools	Validation Scope
Frontend Unit/Component	Vitest, React Testing Library	UI behavior, reducers, and client service logic.
Frontend Contract	MSW	Contract-level checks for API wrapper behavior and payload handling.
Backend Unit/Integration	xUnit, Moq	Controller and service behavior under success and error conditions.
End-to-End	Playwright	End-user flows from login through dash- board and donation paths.

6.3 Functional Validation Scenarios

The most critical business workflows were tested through a mix of API-level checks and manual end-to-end runs:

Table 6.2: Critical Workflow Validation

Workflow	Validation Type	Expected Outcome
Registration and login	Automated + manual	Valid account creation, token issuance, and protected-route access.
Donation and payment callback	Manual integration	Pending donation updates to completed on successful callback.
Volunteer and reporting modules	API + UI checks	Report submission and status transitions operate correctly.
Testimonial moderation and analytics	API tests	Sentiment analysis and moderation states are persisted and retrievable.

6.4 ML Endpoint Validation

Each ML capability is exposed through a dedicated API endpoint in `MLController`. During evaluation, we hit each endpoint with representative data and verified the outputs:

- **Forecasting:** Weekly donation forecasts with confidence intervals.
- **Sentiment:** Text scoring and class mapping (positive/neutral/negative).
- **Donor Churn:** Risk scoring based on donor activity history.
- **Campaign Success:** Probability estimate for campaign goal completion.
- **Anomaly Detection:** Outlier reporting for campaign donation streams.

6.5 Limitations and What Comes Next

This round of evaluation focused mainly on whether things work correctly and whether regressions are caught. What it does not yet cover is performance under load because we have not run formal benchmarks in a production-like environment. The ML models would also benefit from validation against larger historical datasets, which were not available during the project timeline.

Chapter 7

Conclusion and Future Work

Chapter 7: Conclusion and Future Work

7.1 Conclusion

Looking back at what we set out to build, the Donation Management System has turned into a fairly full-featured platform. It covers the core operations that any campus-based charity effort would need including collecting donations, running campaigns, coordinating volunteers, and keeping financial records transparent while also adding some capabilities that existing solutions tend to miss, like in-kind donation tracking with OTP verification and ML-based analytics.

A quick summary of what the system delivers:

- **Donation Processing:** Secure, real-time payments through SSLCommerz with automatic receipt generation and campaign total updates.
- **Campaign Visibility:** Live progress tracking, published updates, and a public reserve fund view so that donors can see exactly where money goes.
- **Volunteer Support:** Activity logging, hour tracking, and a ranking system that rewards consistent contributors.
- **ML-Driven Insights:** Donation forecasting, anomaly detection, donor churn scoring, and sentiment analysis all running inside the backend without external dependencies.
- **Financial Controls:** Expense vouchers require approval before funds are released, withdrawals go through a formal request pipeline, and everything leaves an audit trail.
- **Physical Donation Tracking:** In-kind contributions are documented, valued, and linked to the ledger.
- **Testimonial Moderation:** Automated sentiment scoring flags risky content for admin review before publication.
- **Access Control:** RBAC makes sure each user type (admin, donor, volunteer, campaign creator) sees only what they should.

- **Cross-Device Access:** The UI works on phones, tablets, and desktops, following accessibility guidelines throughout.

The technology stack of ASP.NET Core, React with TypeScript, SQL Server, and Redux Toolkit has held up well. It is mature enough for a production deployment and flexible enough to grow. Security was baked in early: JWT authentication, BCrypt password hashing, server-side input validation, and CORS restrictions are all in place. None of this is exotic, but it covers the OWASP essentials and goes a long way toward earning donor trust.

If nothing else, this project has shown that the manual, spreadsheet-driven approach to campus fundraising can realistically be replaced with software. The result is more transparent, less error-prone, and honestly just easier for everyone involved.

7.2 Future Work

The current system is functional, but there is plenty of room to grow. Some directions worth pursuing:

- **Multilingual Support:** Translating the interface and notification messages into Bangla and other languages would make the platform more accessible. Sentiment analysis would also need to work across languages.
- **Better ML Models:** The current models are a solid starting point, but training on larger, real-world datasets and experimenting with deeper architectures (neural networks, for example) could improve forecast accuracy and churn prediction.
- **Blockchain Transparency:** A distributed ledger layer could provide tamper-proof donation records, which is increasingly expected in the nonprofit space.
- **Personalized Recommendations:** Showing donors campaigns that match their interests, based on past giving patterns, would likely boost repeat donations.

- **ERP Integration:** Universities and large organizations often run ERP systems. Connecting the DMS to those systems would avoid double data entry for financial records.
- **Native Mobile Apps:** A dedicated Android and iOS app would give a better mobile experience and could support offline donation logging for field volunteers.
- **Custom Reports:** Letting admins build their own queries and export data in various formats would reduce the need for manual Excel work.
- **Social Impact Metrics:** Implementing a Social Return on Investment (SROI) framework would let stakeholders see the societal impact of campaigns, not just the financial totals.

7.2.1 Suggested Roadmap

Rolling these out in phases keeps the work manageable:

Table 7.1: Future Enhancement Roadmap

Phase	Primary Deliverables	Expected Outcome
Phase 1 (0–3 months)	Security hardening, expanded test automation, and observability dashboards.	Improved reliability and reduced regression risk in production releases.
Phase 2 (3–6 months)	Mobile optimization, multilingual support, and reporting enhancements.	Higher accessibility and better stakeholder communication across user groups.
Phase 3 (6–12 months)	Advanced personalization and stronger ML models using larger historical datasets.	Better donor retention and improved forecasting quality.
Phase 4 (12+ months)	External ERP integrations and optional transparency-ledger extensions.	Stronger institutional interoperability and long-term governance maturity.

Each phase builds on the previous one, so the system gets more capable over time without needing a risky “big bang” release.

References

References

- [1] David Robinson. Digital fundraising: How technology is transforming charitable giving. *Journal of Nonprofit and Public Sector Marketing*, 31(4):377–392, 2019.
- [2] Jutta Sander. The digital transformation of fundraising: Key technology trends and implications for nonprofit organizations. *Nonprofit Management and Leadership*, 32(1):15–35, 2021.
- [3] Md Hossain et al. E-payment adoption in bangladesh: A study on consumer behavior. *International Journal of Business and Management*, 2019.
- [4] Adam Freeman. *Pro React 18 with TypeScript: Building Enterprise-Grade Applications*. Apress, New York, NY, 2021.
- [5] Boris Cherny. *Programming TypeScript: Making Your JavaScript Applications Scale*. O’Reilly Media, 2019.
- [6] Andrew Lock. *ASP.NET Core in Action*. Manning Publications, Shelter Island, NY, 2nd edition, 2021.
- [7] Mark Masse. *REST API Design Rulebook: Designing Consistent RESTful Web API Interfaces*. O’Reilly Media, Inc., 2011.
- [8] Julie Lerman and Rowan Miller. *Programming Entity Framework Core: Data Access for .NET Applications*. O’Reilly Media, Sebastopol, CA, 2020.
- [9] Carlos Coronel and Steven Morris. *Database Systems: Design, Implementation, & Management*. Cengage Learning, 2016.
- [10] Ravi S Sandhu, Edward J Coyne, Hal L Feinstein, and Charles E Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.

- [11] Ajay Kumar Rathore and P. Vigneswara Ilavarasan. Digital platforms for volunteer management: Enhancing engagement and retention in nonprofit organizations. *VOLUNTAS: International Journal of Voluntary and Nonprofit Organizations*, 32(4):891–906, 2021.
- [12] Robert Jones and Sarah Miller. Modern web development stacks for enterprise applications: A comparison of .net, react, and angular. *Journal of Software Engineering and Applications*, 16(4):135–150, 2023.
- [13] Lin Wang and Wei Zhang. Enhancing donor engagement and retention through personalized digital experience. In *Proceedings of the International Conference on E-business and E-commerce (ICEC)*, pages 55–62. ACM, 2021.
- [14] Christopher Barnes. The rise of crowdfunding and digital fundraising: A nonprofit’s guide to success. *Journal of Nonprofit and Public Sector Marketing*, 30(4):430–445, 2018.
- [15] OWASP Foundation. Owasp top ten web application security risks. *Open Web Application Security Project*, 2021. Available at: <https://owasp.org/Top10>.
- [16] Georg Disterer. Information security management: Iso/iec 27001 standard and its application in software systems. *Journal of Information Security*, 4(2):92–100, 2013.
- [17] Emmanuel Kalinda and Wilfred Mwase. University-based donation platforms: Challenges and opportunities for digital transformation in higher education philanthropy. *Journal of Higher Education Development*, 15(2):88–104, 2022.
- [18] As-Sunnah Foundation. As-sunnah foundation: Faith-based humanitarian digital ecosystem, 2025. Faith-based humanitarian organization utilizing mobile financial services and visual documentation for philanthropic transparency within South Asian communities.
- [19] Bodo B Schlegelmilch, Elfriede Penz, and Meltem Kizilgun. *Strategic International Fundraising*. Routledge, 2022. Focuses on the role of digital platforms in global giving.

- [20] Jianping Huang, Yujia Lu, Kai Chen, Min Zheng, and Jinlong Liu. An online donation system based on blockchain and smart contract. In *2020 IEEE International Conference on Services Computing (SCC)*, pages 102–109. IEEE, 2020.
- [21] Juan Sanchez, Ricardo Gutierrez, and Isabel Martinez. A conceptual framework for implementing secure and transparent e-donation systems. *International Journal of Computer Science and Information Security*, 17(7):112–120, 2019.
- [22] Clayton Hutto and Eric Gilbert. VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 8, pages 216–225. AAAI Press, 2014.
- [23] Alex Banks and Eve Porcello. *Learning React: Functional Web Development with React and Redux*. O’Reilly Media, 2017.
- [24] Sanna Suoranta and Antero Toivonen. Otp-based multi-factor authentication: Security challenges and implementation strategies in web applications. *Journal of Internet Security and Privacy*, 7(1):12–28, 2019.
- [25] Michael B Jones, John Bradley, and Nat Sakimura. Json web token (jwt). RFC 7519, Internet Engineering Task Force, 2015. Available at: <https://tools.ietf.org/html/rfc7519>.
- [26] Niels Provos and David Mazieres. A future-adaptable password scheme. In *USENIX Annual Technical Conference Proceedings*, pages 81–91. USENIX Association, 1999.
- [27] Adam Barth, Collin Jackson, and John C Mitchell. Robust defenses for cross-site request forgery. In *Proceedings of the 15th ACM conference on Computer and communications security*, pages 75–88, 2008.
- [28] Abigail Smith. Tech-enabled volunteer management: Optimizing coordination and reporting for nonprofits. *Volunteer Management Journal*, 25(3):78–90, 2020.

- [29] Wei Chen and Xiaoming Liu. Machine learning applications in nonprofit fundraising: Predictive analytics for donor behaviour. *Journal of Nonprofit Technology and Innovation*, 8(2):45–67, 2022.
- [30] Priya Gupta and Amit Sharma. Predicting donor attrition in charitable organizations using supervised machine learning. *International Journal of Data Science and Analytics*, 10(3):211–225, 2020.
- [31] Hossein Hassani. Singular spectrum analysis: methodology and comparison. *Journal of Data Science*, 5(2):239–257, 2007.
- [32] Salman Ahmed and Bilal Khan. Anomaly detection in financial transactions using isolation forest and singular spectrum analysis. In *Proceedings of the International Conference on Financial Data Engineering*, pages 88–96. IEEE, 2021.
- [33] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422. IEEE, 2008.
- [34] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Martin Fowler, et al. Manifesto for agile software development. *Agile Alliance*, 2001. Available at: <https://agilemanifesto.org>.
- [35] Jim Highsmith and Martin Fowler. The agile manifesto. *Software Development Magazine*, 9(8):29–30, 2001.